



**TUNKU ABDUL RAHMAN UNIVERSITY OF
MANAGEMENT AND TECHNOLOGY**

FACULTY OF COMPUTING AND INFORMATION TECHNOLOGY (FOCS)

ACADEMIC YEAR 2023/2024

BACS3013 DATA SCIENCE

ASSIGNMENT

COURSE NAME	:	Data Science
COURSE CODE	:	BACS3013
SESSION	:	202401
SUBMISSION DEADLINE	:	16 September 2024 (Monday, 10:00 am)
PRESENTATION	:	Weeks 12, 13 and 14
TOTAL MARK	:	100%
WEIGHTAGE TO FINAL MARK	:	42%
Group	:	3 members

Assignment	Marks
TOTAL (100%)	



FACULTY OF COMPUTING AND INFORMATION TECHNOLOGY

Programme: RDS (Group: 4)

Assignment

BACS3013 Data Science

Name(Block Letters)	Registration No.	Signature	Marks
1. CASON SOH KAI SHENG	24WMR08849	<i>Cason</i>	
2. HEA HUI MIN	24WMR08865	<i>Min</i>	
3. ETHEL NG YI YAN	24WMR08861	<i>Ethel</i>	

Tutor's Name: MS. FATIN IZZATI BINTI RAMLI

Date of Submission: 16 September 2024

BACS3013 Data Science

CLO1 - Apply appropriate data science concepts, statistics, and machine learning to make predictions on data.

CLO2 - Practice exploratory data analysis and visualization methods with the aid of appropriate data science tools.

CLO3 - Analyze relevant techniques, statistical methods, and machine learning algorithms for data analytics.

Criterion	Missing or Unacceptable (0-5 mark)	Poor (6-10 marks)	Good (11-15 marks)	Excellent (16-20 marks)	Marks
Data Preprocessing (CLO3)	The data preprocessing step is missing or very poor. Irrelevant, inaccurate, or inappropriate information.	The data preprocessing steps are not listed and described clearly.	The data preprocessing steps are relevant, offering details about the data science components.	The data preprocessing steps introduction are informative, succinct, and offer sufficiently specific details about the data science components.	
Descriptive Analytics and Exploratory Data Analytics (CLO2)	Analytical methods and results are not properly explained. The explanations are not aligned with the data science and business intelligence.	Analytical methods and results are explained. However, the explanations are confusing, incomplete or lacked relevance to the data science and business intelligence.	Analytical methods and results are explained. The explanations are appropriate and related to the data science and business intelligence.	Analytical methods and results are explained well. The explanations are clear, structured, appropriate and related to the data science and business intelligence.	
Graphing and Visualization (CLO2)	Graphing and visualization are omitted or not irrelevant.	Limited graphing and visualization are presented. The information is unclear or	Graphing and visualization are presented. The information is clear.	Graphing and visualization are presented systematically. The information is comprehensive and	

		confusing.		informative. All graphs and visualizations are linked to EDA and advanced analytics appropriately.	
Advanced Analytics and Discussion (CLO3)	Discussions are omitted or confusing. No or very little advanced analytics.	Little discussions are presented. Advanced analytics on the results are unclear or confusing.	Discussions of the results are presented. Advanced analytics on the results are presented.	The significance of the results of the work is discussed, sufficiently inclusive of the advanced analytics. Limitations and future improvements of the studies are identified.	
Model Selection and Report Structure (CLO1)	No justification on the model selection. The structure of the report is incomprehensible, irrelevant, or confusing. Transition from one to another is awkward.	Little justifications are presented on the model selection. The structure of the is weak. Transition from one to another is weak and sometimes difficult to understand.	Adequate justifications are presented on the model selection. The structure of the report is good. Transition from one to another is smooth to show the data science concepts.	Strong justifications are presented on the model selection. The structure of the report is excellent. Transition from one to another is smooth and organized to show the data science concepts.	

Table of Content

1.0 Business Understanding	9
1.1 Business Background	9
1.2 Business Aim	9
1.3 Business Objectives	10
1.4 Inventory of Resources	11
1.5 Requirement, Assumptions and Constraints	12
1.5.1 Requirements	12
1.5.2 Assumptions	12
1.5.3 Constraints	12
1.6 Risk & Contingencies	14
1.7 Data Mining Goals	15
1.8 Data Mining Success Criteria	15
1.9 Project Plan	16
1.10 Motivation	17
2.0 Data Understanding	18
2.1 Data Collection	18
2.2 Data Description	19
2.2.1 Data Loading	19
2.2.2 Basic Data Understanding	20
2.2.3 Dummy Values from Basic Data	21
2.2.4 Descriptive Statistics	22
2.3 Data Exploration	24
2.3.1 Relationship Exploration and Visualisation Using Heatmap	24
2.3.1.1 Scatter Plot of Unit Price Over Sales	25
2.3.2 Distribution Exploration and Visualisation using Pie Chart and Bar Chart	26
2.3.2.1 Transaction Status Distribution	26
2.3.2.2 Bar Chart of Transaction made by Day of Week	27
2.3.2.3 Bar Chart of Number of Transaction Made According to Hour	28
2.3.3 Trends and Pattern Exploration and Visualisation using Line Chart	29
2.3.3.1 Line Graph of Number of Invoices over Time	29
2.3.3.2 Line Chart of Sales over Time	30
2.3.3.3 Bar Chart of Top 10 Items Sold by Quantity	31
2.3.3.4 Bar Chart of Top 10 Items Sold by Sales	32
2.3.3.5 Bar Chart of Top 10 Sales by Customer	33
2.3.3.6 Bar Chart of Top 5 Sales by Country	34
2.3.3.7 Bar Chart of Bottom 5 Sales Made by Country	35
2.4 Data Quality Verification	36
2.4.1 Detect of Outliers	36

2.4.2 Verification of Data Types	38
2.4.3 Detect of Missing Values	39
2.4.4 Detect of Duplicate Rows	40
2.4.5 Verify and Validate Numeric Ranges	41
2.4.5.1 Validate using Quantity	41
2.4.5.2 Validate using Unit Price	42
2.4.5.3 Validate using Sales	42
3.0 Data Preparation	43
3.1 Data Selection	44
3.1.1 Rationale of Inclusion	45
3.1.2 Summary of Selected Data	45
3.2 Data Cleaning	46
3.2.1 Deleting Values	47
3.2.1.1 Remove Quantity With Negative Values	47
3.2.1.2 Remove Unit Price With Negative Values	48
3.2.1.3 Remove StockCode with values of Dot and Post	49
3.2.1.4 Remove Description with Null Value	50
3.2.1.5 Remove the outlier	51
3.2.2 Replacing Values	52
3.2.1 Randomly Add New Customer ID Based On Invoice No	52
3.3 Data Construction	53
3.4 Data Normalisation	55
3.5 Data Concatenation	57
4.0 Modelling	58
4.1 Train Test Split	59
4.2 Evaluation Metrics	60
4.2.1 Evaluation Metrics for Supervised Learning	61
4.2.1.0 Mean Square Error (MSE)	61
4.2.1.1 Mean Absolute Percentage Error (MAPE)	63
4.2.1.2 R-Squared (Coefficient of Determination)	65
4.2.1.3 Root Mean Square Error	67
4.2.1.4 Mean Absolute Error (MAE)	68
4.4 Linear Regression Algorithm	70
4.4.1 Algorithms and Assumptions	70
Algorithms	70
Assumptions of Linear Regression Algorithms	70
4.4.2 Model Building	71
4.4.3 Model Evaluation	73
Mean Square Error	73
Mean Absolute Percentage Error	73

R-squared	73
Root Mean Square Error	73
Mean Absolute Error	74
4.4.4 Hyperparameter Tuning	75
4.4.5 Mode Re-evaluation	76
4.4 Decision Tree Algorithm	77
4.4.1 Algorithms and Assumptions	77
Algorithms	77
Assumptions of Decision Tree Algorithms	77
4.4.2 Model Building	78
4.4.3 Model Evaluation	80
Mean Square Error	80
Mean Absolute Percentage Error	80
R-squared	80
Root Mean Square Error	80
Mean Absolute Error	80
4.5 Random Forest Regression	81
4.5.1 Algorithms and Assumptions	81
Algorithms	81
Assumptions of Linear Random Forest Regression Algorithms	81
4.5.2 Model Building	82
4.5.3 Model Evaluation	84
Mean Square Error	84
Mean Absolute Percentage Error	84
R-squared	84
Root Mean Square Error	84
Mean Absolute Error	84
5.0 Evaluation	85
5.1 Comparisons	85
5.2 Best Model Performance	85
6.0 Deployment	86
7.0 Conclusions	87
Linear Regression Model	87
Decision Tree Model	87
Random Forest Model	88
8.0 References	89
Assignment[Final].ipynb	89
10.0 Appendix	90
10.1.0 Coding Part for Data Understanding	90
10.1.1 Data Loading	90

10.1.2 Descriptive Statistics	90
10.1.3 Data Exploration	91
10.1.3.1 Relationship Exploration and Visualization using Heatmap	91
10.1.3.2 Scatter Plot of Unit Price over Sales	91
10.1.3.3 Distribution Exploration and Visualisation using Pie Chart	92
10.1.3.4 Bar Chart of Transaction made by Day of Week	92
10.1.3.5 Bar Chart of Number of Transaction Made According to Hour	93
10.1.3.6 Line Graph of Number of Invoices over Time	93
10.1.3.7 Line Chart of Sales over Time	94
10.1.3.8 Bar Chart of Top 10 Items Sold by Quantity	94
10.1.3.9 Bar Chart of Top 10 Items Sold by Sales	95
10.1.3.10 Bar Chart of Top 10 Sales by Customer	95
10.1.3.11 Bar Chart of Top 5 Sales by Country	96
10.1.3.12 Bar Chart of Bottom 5 Sales by Country	96
10.1.4 Data Quality Verification	97
10.1.4.1 Detect of Outliers	97
10.1.4.2 Verification of Data Types	97
10.1.4.3 Detect of Missing Values	97
10.1.4.4 Detect of Duplicate Rows	97
10.1.4.5 Verify and Validate Numeric Ranges	98
10.1.5 Data Preparation	98
10.1.5.1 Data Selection	98
10.1.5.2 Data Cleaning	99
10.1.5.2.1 Clean quantity negative number	99
10.1.5.2.2 Clean Unit Price	99
10.1.5.2.3 Clean Dot, Post and Description is empty	99
10.1.5.2.4 Replace Customer ID	100
10.1.5.2.5 Remove Outlier	103
10.1.6 Data Construction	104
10.1.6.1 Separate Categorical and continuous data	104
10.1.8 Data Normalisation	105
10.1.9 Data Concatenation	105
10.1.10 Data Modeling	106
10.1.10.1 Linear Regression	106
10.1.10.1.1 Hyperparameter Tuning	107
10.1.10.1.2 Model Revaluation	107
10.1.10.2 Decision Tree	108
10.1.10.3 Random Forest	110
10.1.10.4 Compare R^2	111

1.0 Business Understanding

1.1 Business Background

Find Me a Gift is an online retail store that was established in the year of 2000 and is based in the United Kingdom. The idea of this website is given by three friends where a sudden idea pops out from their brain to create happiness and joy to people. Find Me a Gift online store is taken-on by Adam and Lena. Its primary service is to provide a convenience to the users or customers to purchase the gift to their family, friends or even their couples during occasions as they provide doorstep delivery of the purchased items. It was started as a small online retailer becoming a major eCommerce gift business serving over 8 Million customers in the last 20 years specialising in providing a wide area of unique and quirky gifts for various occasions and events such as Christmas and Birthday. In the year of 2019 the ownership of Find Me a Gift was handed to Days of Your Life groups with the expanded product ranges to include not only occasional gifts but also personalised gifts, gadgets and novelty products. Find Me a Gift has also extended its business where customers can find their products from Boots, Primark, Debenhams as well as Amazon besides their official website.

1.2 Business Aim

The “Find Me a Gift” online retail store aims to utilise the available dataset to come up with data-driven strategic decisions, with the primary focus on predicting future sales. By leveraging techniques in data science, the store plans to gain insights into customer behaviour, product performance and market trends, all of which will be used to support accurate sales forecasting. These would allow the online store to be able to enhance customer satisfaction and personalization while also to guide inventory management and marketing strategies. The ultimate goal is to improve its overall efficiency, streamline product offerings and improve overall customer satisfaction through personalization in order to solidify the store’s competitive position in the online retail market. This would lead to the increase of sales and revenue for the online retail store. By effectively using the dataset to predict sales trends, the store will be able to anticipate customer needs and make informed decisions which will help in contributing to the retail store’s long-term business growth.

1.3 Business Objectives

One of the first measurable actions that could be taken is to use historical sales data to develop predictive models in order to forecast future sales trends. This would allow the online retail store to understand the difference in impact of different possible decisions in terms of stock levels and marketing campaigns. The next objective would be to leverage sales predictions to optimise the store's inventory. Through this way, the online retail store is able to ensure that high-demand products are always sufficiently stocked while minimising excess inventory of items that are in low demand to reduce costs.

In addition, the online retail store is able to use the predicted sales data to analyse and plan out future budget allocations for different departments so that all resources are mainly focused on high-return areas based on expected revenue trends. Another business objective to be mentioned is to have expansion and growth planning. The online retail store is able to make long-term decisions based on predictive sales in order to decide on expansion of new product lines or entering new markets.

Additionally, another measurable action that will be taken is to optimise dynamic pricing strategies by utilising sales prediction. The online retail shop will adjust prices based on anticipated demand and customer behaviour to maximise profit margins. Sales prediction can also be shared to suppliers and vendors to improve coordination through adjusting production schedules and delivery timelines which in the end improves overall performance of the online retail shop.

1.4 Inventory of Resources

The inventory of resources typically refers to a comprehensive list or assessment of available assets and materials within an organisation or project. Resources used to complete this assignment can be categorised in 3 groups, which are the human resources, the intellectual resources as well as physical and technological resources.

Firstly, the project's human resource consists of project members and university staff. The project members include Cason Soh Kai Sheng, Ethel Ng Yi Yan, Hea Hui Min. These individuals are the students of Year 2 semester 1 from group 4, pursuing a Bachelor of Computer Science (Honors) in Data Science at Tunku Abdul Rahman University of Technology and Management (TARUMT). Overseeing the project, Cik Fatin Izzati Binti Ramli, a dedicated lecturer and tutor for our data science subject at TARUMT, provided invaluable guidance to the project members throughout the project completion.

Regarding the intellectual resources, the dataset utilised for this project originates from the UC Irvine Machine Learning repository website, which is a platform for a set of databases, domain theories, and data generators utilised by the machine learning community for conducting empirical analyses of machine learning algorithms among data scientists and machine learning experts. Project members also leverage web browser platforms like Google Chrome and Microsoft Edges to explore various websites such as W3schools, GreekforGreek as well as Github to gain information and knowledge in order to perform this project.

In the terms of physical and technological resources, project members employed their personal laptops or desktops for project development. In terms of software, the coding process was facilitated by the Jupyter Notebook, an interactive web-based computational platform. As for the documentation, we have utilised Google Docs, which is known as an online word processing tool which allows all of the project members to collaborate and amend the documentation simultaneously. Essential resources such as the datasets and the Jupyter Notebook, Visual Studio and Google Collab that will be used to develop graphs or models are stored into Google Drive, which is a service for file storage and sharing. Throughout the project's duration, project members have used the Whatsapp call feature and Google Meet in order to converge meetings and communications on the project that is specified as it provides the convenience to communicate with each other without boundaries. Lastly, the Google Classroom is also utilised for instructors to deliver the project to them and the submission of the project before the due date.

1.5 Requirement, Assumptions and Constraints

1.5.1 Requirements

In order to finish the project successfully, we must meet several key requirements. Firstly, we should create a detailed project plan that includes tasks, timelines and resources in order to ensure that our project is done smoothly within the given time. Project planning can also minimise the probability of delaying the project timeline or duration which may lead to over budget and resources. Besides that, we assign roles and responsibilities to all team members in our project members to make sure that everyone involved in the project promotes efficiency and effectiveness. We will also utilise appropriate tools in order to perform our tasks. For example, we will be utilising the Python in Jupyter Notebook and its relevant libraries for coding. In addition, we should also ensure the resources such as the dataset that has been used is free of unrestricted access to prevent any copyright issues. We will be using a dataset from the UC Irvine Machine Learning repository that is accessible to any parties without any concerns of copyright related issues.

1.5.2 Assumptions

Within the project, we should also underlying a few assumptions. First of all, the dataset is assumed to be accurately recorded. Besides that, it is assumed that all necessary tasks have been carried out correctly, resulting in accurate results. Lastly, the dataset downloaded from the UC Irvine Machine Learning repository website is assumed to be an unmodified real world dataset.

For the assignment, we assume that the same invoice number is assigned to only one particular customer. We assume that the sales of the transaction is the multiplication of unit price quantity for each product in a transaction.

1.5.3 Constraints

There are two main constraints in this project. First and foremost, the first constraint is the time given for the completion of the project is limited. This project is needed to be completed and submitted by week 12 and we have only around 7 weeks to complete this project. As we need to present our work around week 10, therefore we should have done most of the work before the arrival of week 10. In order to ensure the on time delivery of the project, we should closely adhere to the project plan. Besides the time constraints, we think that the lack of comprehensive knowledge is also one of the constraints that we are facing. This is because the language, python, is not a familiar language to all of our project members, making it difficult for us to understand and have an early start on the project immediately after the assignment has been distributed.

Therefore, additional comprehensive research must be carried out by using the Internet to learn more about the language and its full usage in order to fully grasp the knowledge to perform the task that is required in this assignment.

1.6 Risk & Contingencies

The term of risk in data science refers to the probability of loss or negative impact resulting from the uncertainties in the data processing method. The risk for this assignment is where the dataset that we are going to use may consist of incomplete, inaccurate data where missing values, duplicates and outliers may occur in the data set. An incomplete and inaccurate data may lead to bias and inconsistencies which can undermine the reliability of the final analysis result. Besides that, there may also occur model risk where the dataset has overfitting or underfitting the model. A model will be overfitted if it performs well on the training data but poorly on unseen data while the underfitting model where the model is too simple to capture trends and patterns in the data.

Contingencies can be said as the measurements that should be taken or implemented to deal with unanticipated problems when in some situations might not go as planned. In order to avoid or overcome the potential risk that may happen, data imputation and validation is crucial to ensure data accuracy and consistency. Data imputation such as handling or dealing missing values using replace or delete method to get rid of missing values. Data validation checks should also be done regularly. Besides that, in order to avoid loss of data, we should also ensure that we have regular backups and have a recovery plan in case of data losses or system failure.

1.7 Data Mining Goals

1. Analysing external factors such as weather conditions, holidays or economic indicators which can be incorporated into the sales prediction model in order to improve its performance in predicting spikes and declines in sales of specific products.
2. Understanding the products frequently bought together to optimise product placement, cross-sell opportunities, and promotions.
3. Predicting future sales trends based on historical data to manage inventory, plan marketing campaigns, and adjust pricing strategies.
4. Estimating the total value a customer will bring over their entire relationship with the store to guide investment in customer acquisition and retention strategies.
5. Developing product recommendations engine based on past purchases, browsing behaviour, and other relevant data to enhance customer experience and increase sales.

1.8 Data Mining Success Criteria

Data mining projects' impact and efficacy can be assessed with the use of data mining success criteria. To ensure the success of the analysis:

1. The accuracy of each of the models that we used should be over 80%.
2. The precision which are predicted by the model should exceed 80 %
3. The accuracy of predicting and forecasting the future sales should be at least 80%

1.9 Project Plan

Project Phases	Activities	Duration	Start Time	Finish Time
Business Understanding	- Identify business background - Establish Business Aims - Determine Business Objectives - Discover inventory resources - Examine the requirements, make assumptions and set constraints - Define potential risk and contingencies - Define data mining goals - Determine data mining success criteria - Create a project plan - Describe project motivation	1 week	31 July 2024	6th August 2024
Data Understanding	- Data Collection - Data Description - Data Exploration and Visualisation - Data Quality Verification	6 days	7th August 2024	12th August 2024
Data Preparation	- Data Selection - Data Cleaning - Data Construction - Data Concatenation - Data Normalisation	1 week	13th August 2024	19th August 2024
Modelling	- Train Test Split - Build the model - Evaluate the metrics	1 week 4 days	20th August 2024	30th August 2024
Evaluation	- Evaluate the results of model - Determine the best model	6 days	31th August 2024	5th September 2024
Deployment	- Plan deployment - Produce final report - Summarise the report	7 days	6th September 2024	12th September 2024

1.10 Motivation

This project provides a vital platform for project members to bridge a gap between theoretical knowledge and practical application. Project members are required to apply knowledge and concepts learned in their lectures and practical sessions and put them into a real world dataset, implementing the practical skills and gaining hands-on experiences. By working with the datasets, project members can clearly and deeply understand the data analysis processes such as data cleaning, data preprocessing, model developing and training as well as gain insights and knowledge from the model that has been trained. Project members are able to identify and choose the most suitable model and methods for a specific business problem after implementing various algorithms and techniques. This experiential learning allows project members to develop critical thinking and problem-solving skills which is important to gain success in the data science field.

Apart from that, fulfilling this assignment's requirements shows that the project members have a thorough understanding of the course material and can effectively use the knowledge gained from the materials to finish the project. This assignment holds a significant weightage in project members academic journey and it offers substantial assistance and value for them especially as a junior data scientist. Completing the data science assignment effectively demonstrates to the other project members how committed they are to learning the material, which could result in better grades. So, performing well on data science projects not only validates one's understanding of the field but also puts one in a good position for opportunities in the future.

2.0 Data Understanding

2.1 Data Collection

The screenshot shows the UC Irvine Machine Learning Repository website. The main header includes the repository logo, navigation links (Datasets, Contribute Dataset, About Us), a search bar, and a login button. The 'Online Retail' dataset page is displayed, featuring a blue header with the dataset name and donation date (11/5/2015). Below the header, a description states: 'This is a transnational data set which contains all the transactions occurring between 01/12/2010 and 09/12/2011 for a UK-based and registered non-store online retail.' The page is organized into several sections: 'Dataset Characteristics' (Multivariate, Sequential, Time-Series), 'Subject Area' (Business), 'Associated Tasks' (Classification, Clustering), 'Feature Type' (Integer, Real), '# Instances' (541909), and '# Features' (6). A 'Dataset Information' section provides additional details, including the creator's name (Daqing Chen) and affiliation (School of Engineering, London South Bank University). On the right side, there are buttons for 'DOWNLOAD', 'IMPORT IN PYTHON', and 'CITE', along with statistics (8 citations, 221938 views) and a 'Keywords' section (sales). The 'DOI' (10.24432/CSBW33) is also listed.

Figure 2.1.0

The dataset that we are going to analyse is transactional data related to an online retail store and it is obtained from UC Irvine Machine Learning Repository website which is shown in **Figure 2.1.0**. The transactional dataset contains all the transactions made by a United Kingdom-based online retail shop over the period of one year from 1 December 2010 to 9 December 2011. This dataset, which is contributed by Chen Da Qing from the School of Engineering at London South Bank University, falls within the business domain and it provides valuable insights into retail operations.

The dataset comprises 541,909 observations, making it a substantial resource for analysis, consisting of 8 variables, which are Invoice Number, Stock Code, Item Description, Product quantity that is purchased per transaction, Invoice Date, Unit Price, Customer ID, Country.

2.2 Data Description

2.2.1 Data Loading

To prepare the data for processing, the CSV file titled ‘Online Retail.CSV’ is loaded into the Pandas Dataframes using Jupyter Notebook. The dataset has 541,909 rows, referred to as observations and 8 columns, referred to as variables. *Figure 2.2.1.0* shows the coding segment of *Figure 2.2.1.0* which displays the entire dataset in a tidy format with their respective variables.

```
[55] import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
df = pd.read_csv('OnlineRetail.csv')
df
```

Figure 2.2.1.0

	InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice	CustomerID	Country
0	536365	85123A	WHITE HANGING HEART T-LIGHT HOLDER	6	1/12/2010 8:26	2.55	17850.0	United Kingdom
1	536365	71053	WHITE METAL LANTERN	6	1/12/2010 8:26	3.39	17850.0	United Kingdom
2	536365	84406B	CREAM CUPID HEARTS COAT HANGER	8	1/12/2010 8:26	2.75	17850.0	United Kingdom
3	536365	84029G	KNITTED UNION FLAG HOT WATER BOTTLE	6	1/12/2010 8:26	3.39	17850.0	United Kingdom
4	536365	84029E	RED WOOLLY HOTTIE WHITE HEART.	6	1/12/2010 8:26	3.39	17850.0	United Kingdom
...	...	...	...	...	...	...	...	...
541904	581587	22613	PACK OF 20 SPACEBOY NAPKINS	12	9/12/2011 12:50	0.85	12680.0	France
541905	581587	22899	CHILDREN'S APRON DOLLY GIRL	6	9/12/2011 12:50	2.10	12680.0	France
541906	581587	23254	CHILDRENS CUTLERY DOLLY GIRL	4	9/12/2011 12:50	4.15	12680.0	France
541907	581587	23255	CHILDRENS CUTLERY CIRCUS PARADE	4	9/12/2011 12:50	4.15	12680.0	France
541908	581587	22138	BAKING SET 9 PIECE RETROSPOT	3	9/12/2011 12:50	4.95	12680.0	France

541909 rows × 8 columns

Figure 2.2.1.0

2.2.2 Basic Data Understanding

Variable Name	Types of Data Element	Dependency	Description
InvoiceNo	Categorical - Nominal	Independent variable	6 digit numerical values assigned to each transaction. If The letter 'c' at the start of the code indicates a cancellation
StockCode	Categorical - Nominal	Independent variable	5-digit integral number uniquely assigned to each distinct product
Description	Categorical - Nominal	Independent variable	Product name
Quantity	Integer	Dependent variable	Quantity of each product (item) per transaction
InvoiceDate	Categorical - Nominal	Independent variable	Day and time when each transaction was generated
Unit Price	Continuous	Independent variable	Product price per unit
CustomerID	Categorical - Nominal	Independent variable	5-digit integral number uniquely assigned to each customer
Country	Categorical - Nominal	Independent variable	Name of the country where each customer resides

Table 2.2.2.0

Table 2.2.2.0 shows all the variables' names and their respective types of data element, their dependency and description of each of them.

2.2.3 Dummy Values from Basic Data

Variable Name	Types of Data Element	Dependency	Description
Sales	Continuous	Dependent variable	total revenue generated from unit price and quantity sold
Month	Integer	Independent variable	Value of month that is extracted from the InvoiceDate variable
Year	Integer	Independent variable	Value of year that is extracted from the InvoiceDate variable
month_year	Integer	Independent variable	Value of month and year that is extracted from the InvoiceDate variable (assume the day is 1st of every month)
Hour	Integer	Independent variable	Value of hour that is extracted from the InvoiceDate variable

Table 2.2.3.0

Table 2.2.3.0 shows the made up dummy values with their respective variable name, types of data element, dependency, description.

2.2.4 Descriptive Statistics

Descriptive statistics help us to summarise and describe the main features of a dataset by simplifying large datasets, making it easier to understand and interpret the data. There are two main focuses in descriptive statistics, which are the measure of central tendency and the measure of variability. The measure of central tendency includes mean, median and mode, which represent the centre of the data distribution while the measure of variability involves standard deviation, variance, minimum and maximum which is used to describe the spread or dispersion of the data. Descriptive statistics can be used to summarise a large or huge datasets by using the measurement of central tendency. By using the `data.median()` and `data.mode()` we are able to identify the median and mode of the numerical values in the dataset which can be shown in **Figure 2.2.4.1** and **Figure 2.2.4.2**. Besides that, using the functions `data.describe()`, we are able to view the count, mean, maximum values, standard deviation, minimum values, median, the 1st quartile range and the 3rd quartile range of the given observations which can be referred to in **Figure 2.2.4.0**. The minimum of Quantity and Unit Price variables is shown negative. Since it is negative, the data cleaning process is needed for these variables. This is because the dataset has a negative value. So we need to take out the negative value, because that quantity should not have a value that is lower than one.

Unit Price and Quantity statistics
=====

	UnitPrice	Quantity
count	541909.000000	541909.000000
mean	4.611114	9.552250
std	96.759853	218.081158
min	-11062.060000	-80995.000000
25%	1.250000	1.000000
50%	2.080000	3.000000
75%	4.130000	10.000000
max	38970.000000	80995.000000

Figure 2.2.4.0

Median of Unit Price and Quantity

=====

0	
UnitPrice	2.08
Quantity	3.00

Figure 2.2.4.1

Mode of Unit Price and Quantity

=====

	UnitPrice	Quantity
0	1.25	1

Figure 2.2.4.2

2.3 Data Exploration

2.3.1 Relationship Exploration and Visualisation Using Heatmap



Figure 2.3.1.0

The **Figure 2.3.1.0** above shows the heatmap distribution among the variables that is used or created in order to perform data analysis. Heatmap is used to visualise and identify the correlation or strong relationships between two variables in the dataset. In the heatmap above, we have used the quantity, unit price, sales, year, month and hour variables which may be important in analysing and finding meaningful patterns. Any relationship that is higher than 0.5 is considered a strong relationship while a relationship that has a correlation matrix that is lower than 0.5 is considered a weak relationship. From the heatmap distribution above, we can see that there is a strong relationship between the sales and quantity, which is 0.89. Quantity and the Sales can be said as highly correlated as the sales is dependent on the quantity that has been purchased by a customer. As the quantity of items purchased by a customer has increased, therefore the sales will also increase.

2.3.1.1 Scatter Plot of Unit Price Over Sales

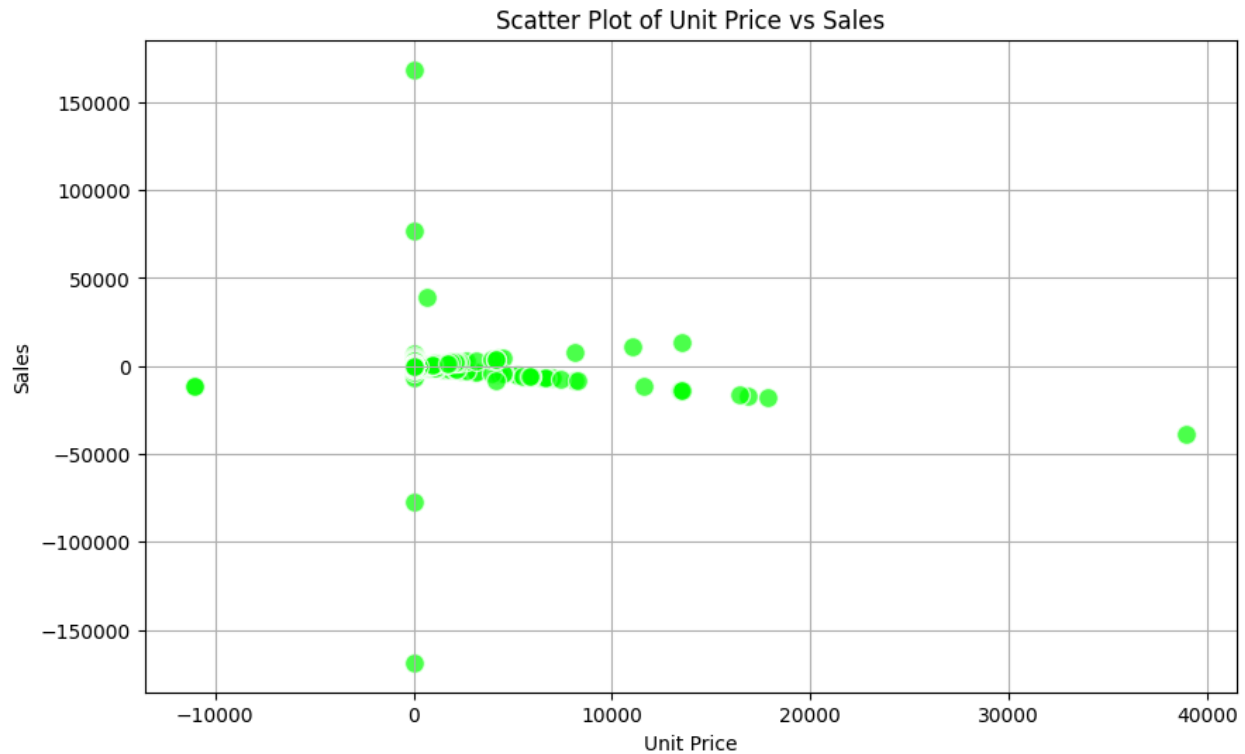


Figure 2.3.1.0

Figure 2.3.1.0 shows the scatter plot distribution of Unit Price and Sales. There are negative values for both Unit Price and Sales which are likely errors or outliers in the dataset. Most of the data points in the scatter plot distribution are clustered near the origin which implies that the majority of Unit Price and Sales are relatively small.

The above statement suggests that further cleaning of the dataset are necessary to remove all the unrealistic and erroneous values.

2.3.2 Distribution Exploration and Visualisation using Pie Chart and Bar Chart

2.3.2.1 Transaction Status Distribution

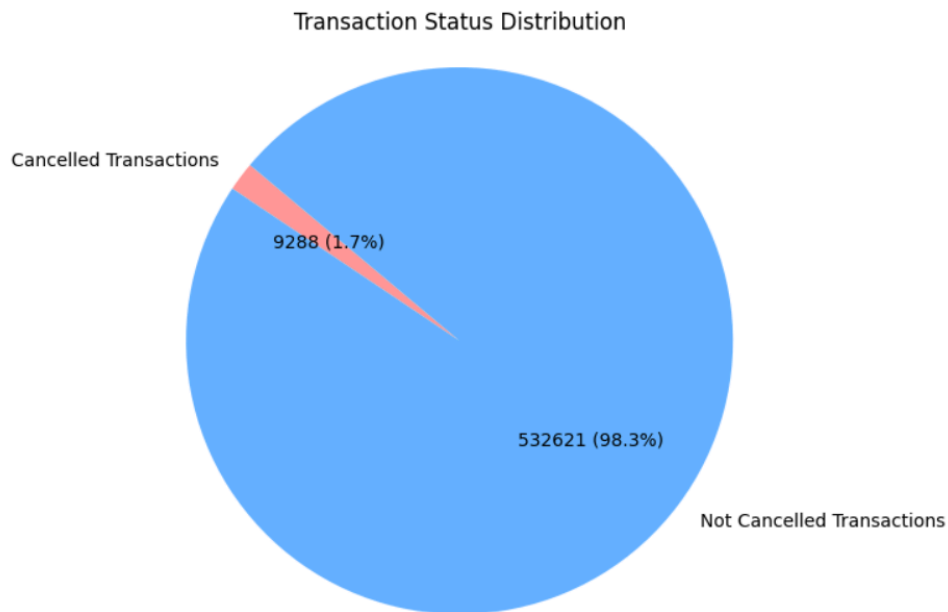


Figure 2.3.2.0

The **Figure 2.3.2.0** above is all about the distribution of the transaction status in all the dataset. There are 9288 observations that have been cancelled which stands around 1.7% among all the transactions that have been made in a year. There are many reasons that may lead to the cancellation of transactions or item purchasing. For example, there may be insufficient amounts in their bank account, incorrect billing information like entering the wrong address, customer change of mind, etc. The cancelled transactions will be removed later since it is not necessary. Besides that, there are also 98.3% which are 532621 non-cancelled transactions, which can be said as the successful trading on the UK online retail store.

2.3.2.2 Bar Chart of Transaction made by Day of Week

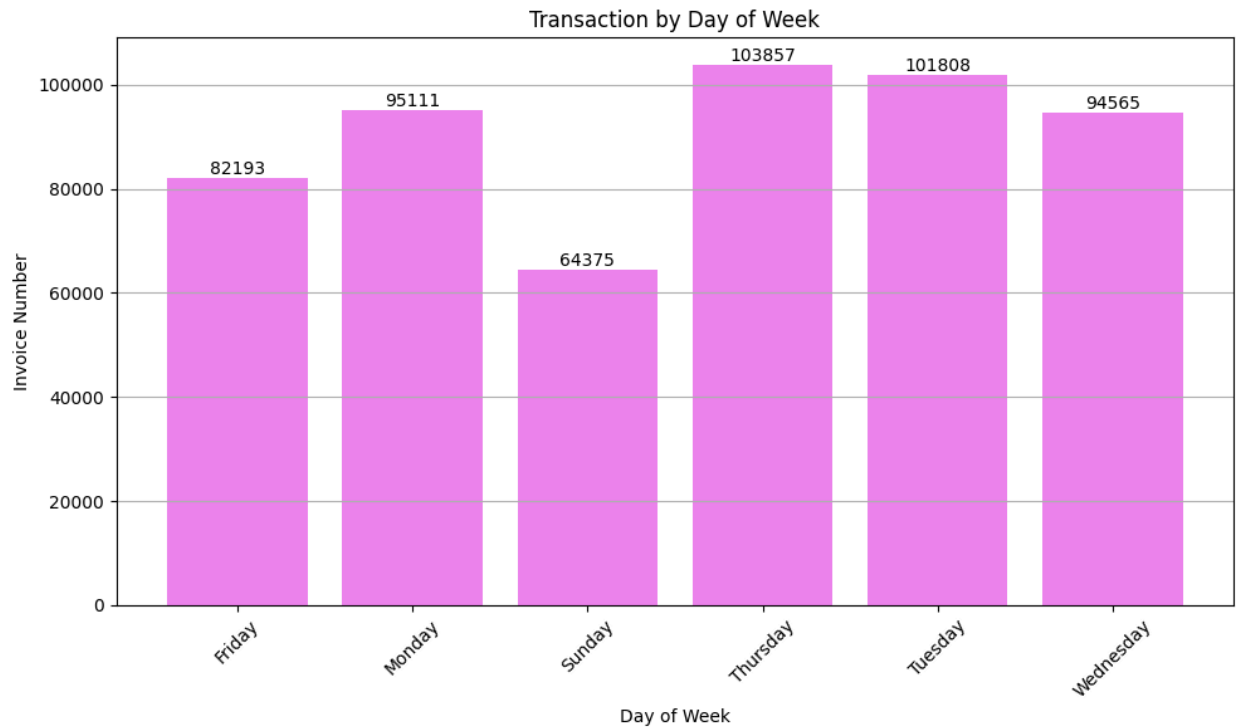


Figure 2.3.2.2.0

The **Figure 2.3.2.2.0** above illustrates the bar chart of transactions made that is categorised using the Day of Week. The x-axis of the graph is the day of week which is Monday to Sunday and the y-axis of the graph is the number of invoices made which have the values from 0 to 10000. As we can observe from the graph, most of the customers tend to make a purchase decision on Thursday with the total number of 103867 invoices generated on that day. The second highest number of invoices generated is on Tuesday which has a number of 101808 transactions made, followed by Monday and Wednesday. However, we can see that there is no transaction made on Saturday which may be because of some filtering during data collection. There is a significant difference in transactions made on Sunday within the week.

2.3.2.3 Bar Chart of Number of Transaction Made According to Hour

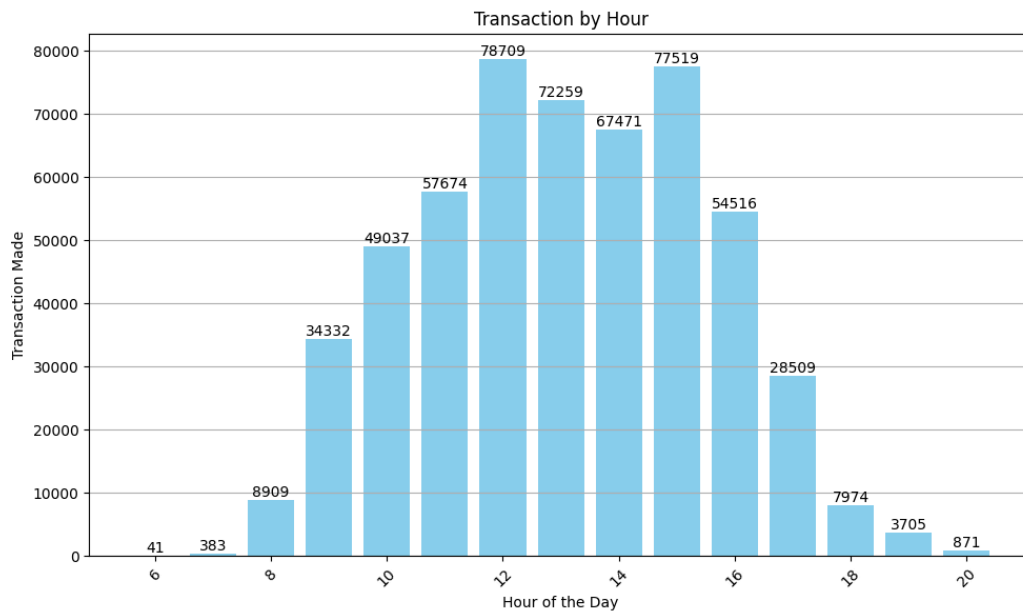


Figure 2.3.2.3.0

The **Figure 2.3.2.3.0** above shows the bar chart of the number of transactions made according to hour. The x-axis consists of hours in military time which is distributed from 00:00 (12am) to 20:00 (8pm) from the graph and the y-axis is distributed from 0 to 80000 number of transaction items made. As we can observe from the above diagram, we are able to know that most of the transaction has been made around the afternoon or we can say around the lunch time from 12 pm to 3pm. The most transactions that have been made is at 12 pm which consists of 78,709 transactions made, followed by 77,519 transactions made at 3pm which can be said just right after the lunch hour and the least transaction made is at 6 am which only consists of 41 transactions.

2.3.3 Trends and Pattern Exploration and Visualisation using Line Chart

2.3.3.1 Line Graph of Number of Invoices over Time

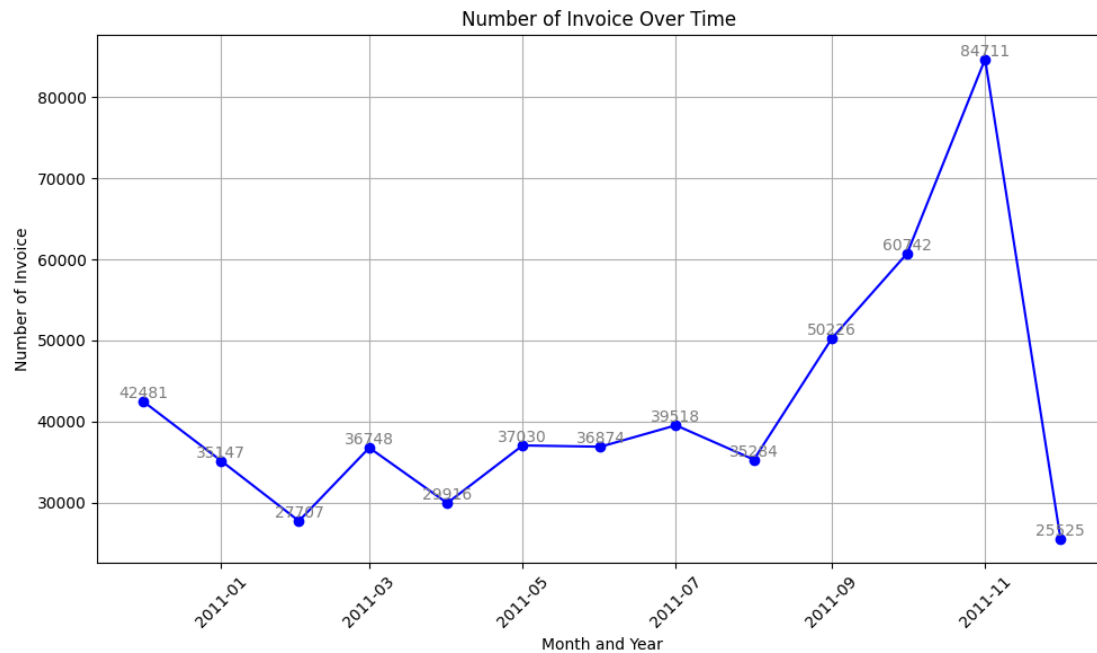


Figure 2.3.3.1.0

The **Figure 2.3.3.1.0** above illustrates the number of invoices (monthly transactions) over time. The x - axis represents the month and year from December 2010 to December 2011, while the y - axis represents the number of transactions from 0 to around 90000 number of invoices a month. The distribution of the line graph is skewed to the left where the maximum count of invoices made is at the month of November 2011 which consists of 84,711 numbers of transactions made. Number of transactions tends to increase from September 2011 to November 2011, reflecting seasonal buying behaviour. In February 2011 had the lowest number of transactions made which had only 27,707 transactions.

2.3.3.2 Line Chart of Sales over Time

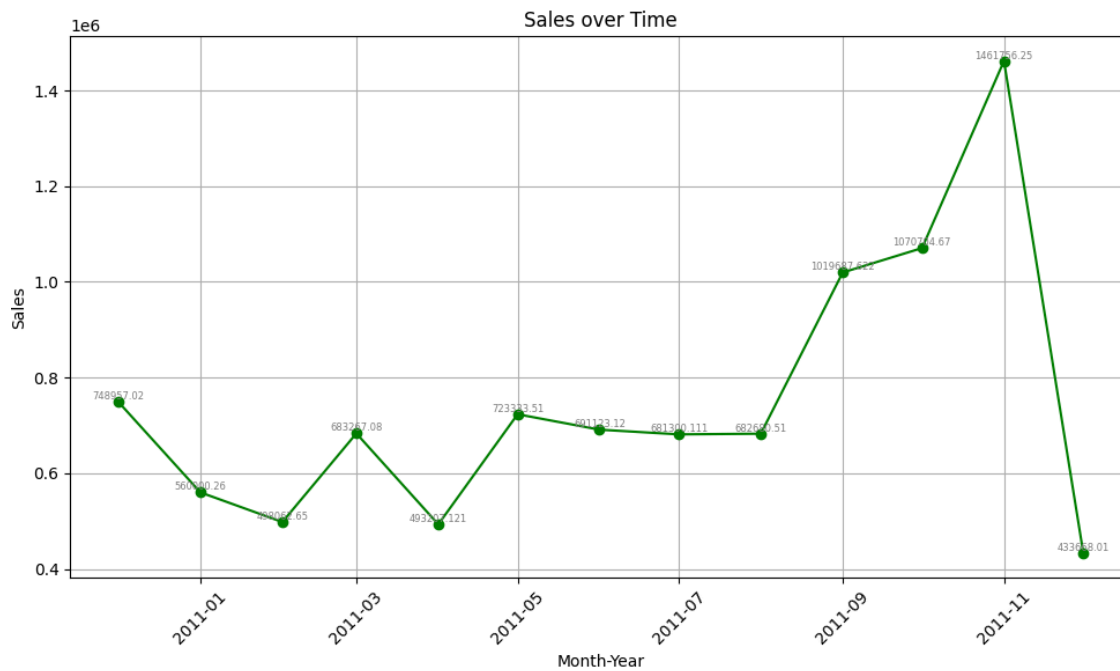


Figure 2.3.3.2.0

The **Figure 2.3.3.2.0** is a line graph which illustrates the total sales made each month. The x-axis represents the month of the year and the y-axis represents the total revenue that has been made for the stated month from total sales from \$400,000 to \$1,400,000. The highest sales that have been made is in November of 2011 which has a total revenue of \$1,461,756.25 followed by the month of October which have contributed a total sales of \$1,070,764.67. The least amount of sales has been made in the month of April 2011 which only contributes \$193204.121. The sales tend to increase around the month of September. The line graph is skewed to the left with most of the sales made around the last quarter of 2011.

2.3.3.3 Bar Chart of Top 10 Items Sold by Quantity

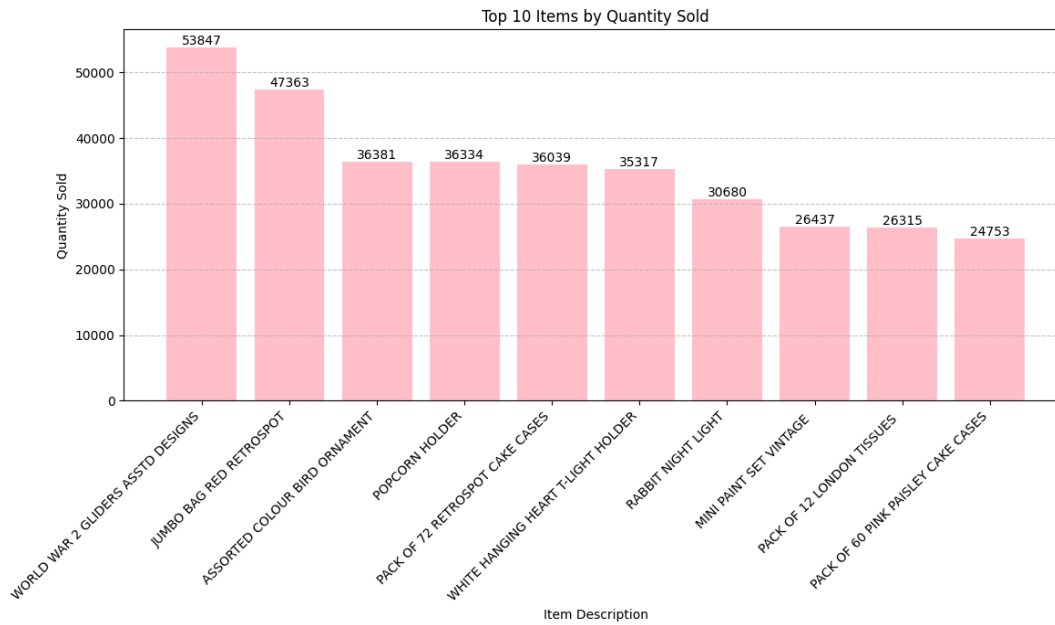


Figure 2.3.3.3.0

The **Figure 2.3.3.3.0** above shows the bar chart of top 10 items sold by quantity. The x-axis is about the description of each item sold while the y-axis is the number of quantities from 0 to 50,000. The item ‘World War 2 Gliders Asstd Designs’ has sold out with the quantity of 53,947 which is the most among all of the items, followed by ‘Jumbo Bag Red Retrospot’ with sold out 47,363 items. Besides that, the ‘Assorted Colour Bird Ornament’, ‘Popcorn Holder’, ‘Retrospot Cake Cases’ and ‘White Hanging Heart T-Light Holder’ has a similar number of quantity sold which are 36,381, 36,334, 36,039 and 35,317 items respectively.

2.3.3.4 Bar Chart of Top 10 Items Sold by Sales

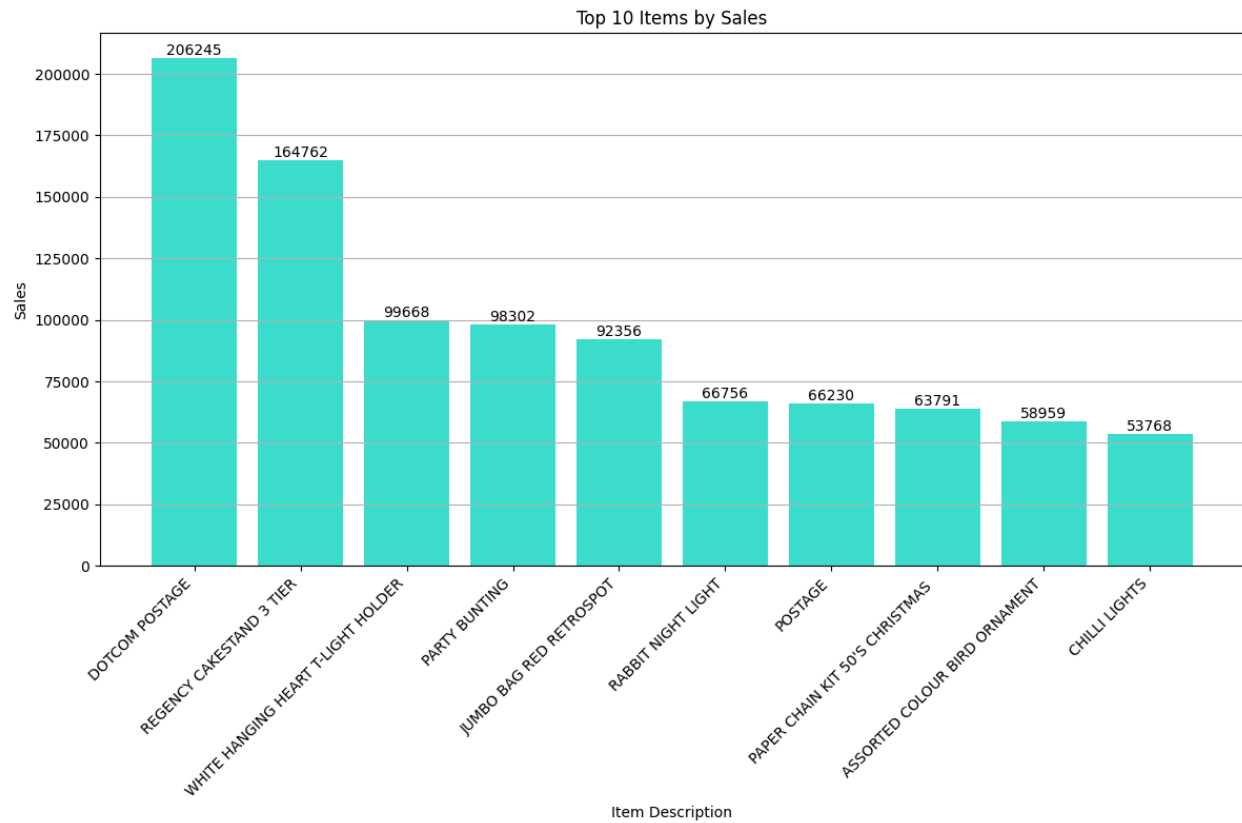


Figure 2.3.3.4.0

The **Figure 2.3.3.4.0** above shows the bar chart which is the illustration of the top 10 items sold by sales in decreasing order from the left to the right. The x-axis represents the item description of the top 10 items sold while the y-axis represents the sales that have been made by selling the items. The “Dotcom Postage” item sold the most in terms of sales which is \$206,245.

2.3.3.5 Bar Chart of Top 10 Sales by Customer

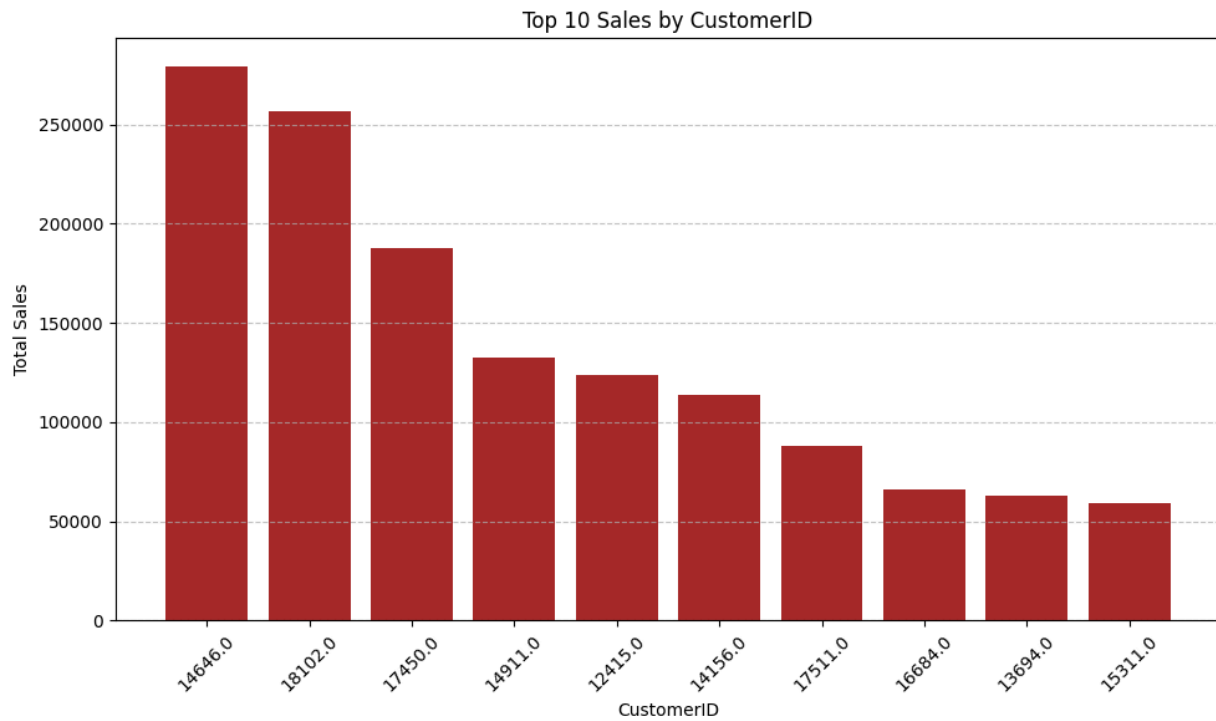


Figure 2.3.3.5.0

The **Figure 2.3.3.5.0** above shows the bar chart which is the illustration of the top 10 sales by customer in decreasing order from the left to the right. The x-axis represents the customerID while the y-axis represents the sales that have been made by each customer. The customerID “14646” has the most sales which is above \$250,000.

2.3.3.6 Bar Chart of Top 5 Sales by Country

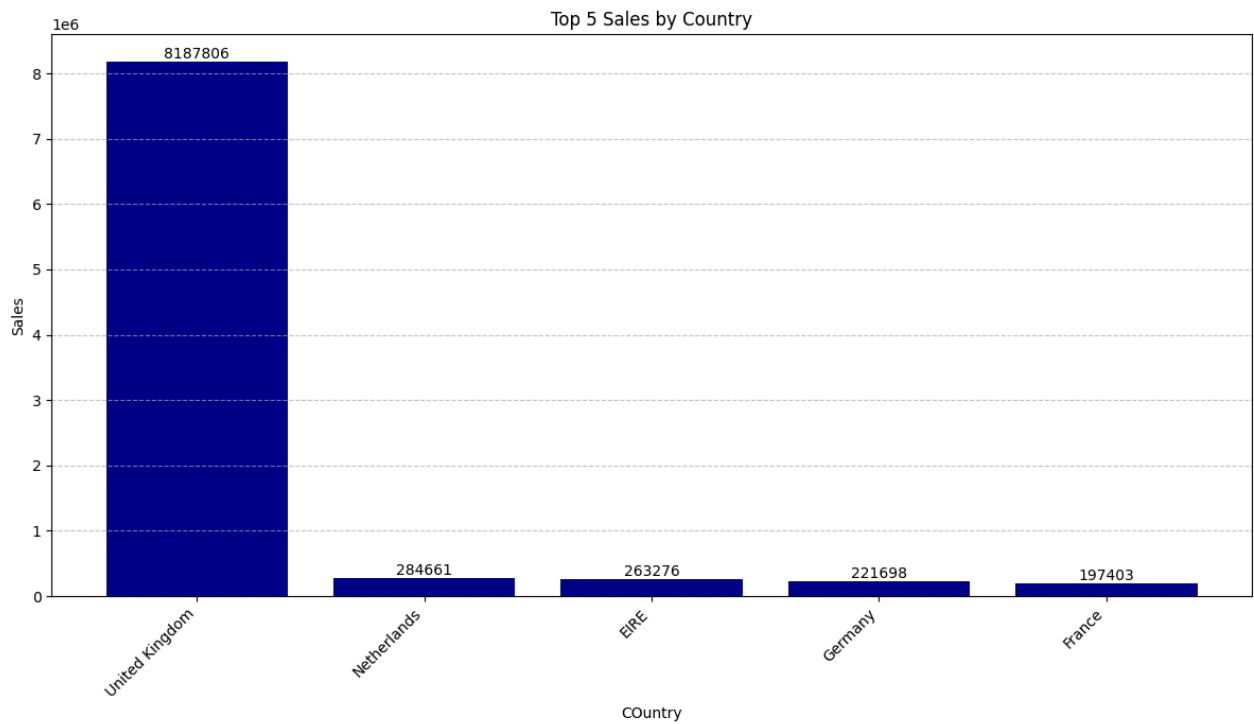


Figure 2.3.3.6.0

The **Figure 2.3.3.6.0** illustrates the bar chart of top 5 sales by country. The x-axis represents the country's name while the y-axis represents the sales that have been made by the countries. We can tell that the United Kingdom produces the most sales out of the 5, which is \$8,187,806.

2.3.3.7 Bar Chart of Bottom 5 Sales Made by Country

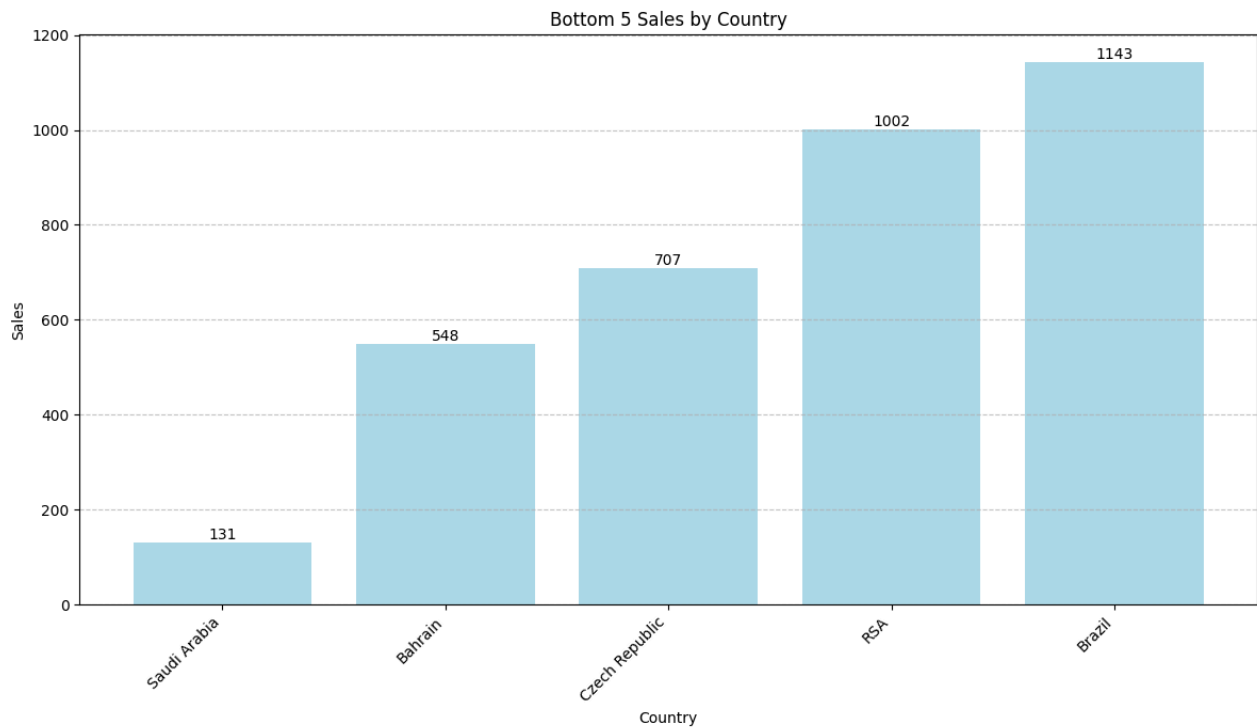


Figure 2.3.3.7.0

The **Figure 2.3.3.7.0** above shows the bottom sales made by country in the online retail store where the x-axis is the name of the country and the y-axis is the sales made. The least sales made are Saudi Arabia with the sales of only \$131 followed by Bahrain with sales of a total of \$548. Besides Saudi Arabia and Bahrain, Czech Republic, RSA and Brazil is also the 5 of the bottom sales made by country which consists of \$707, \$1002 and \$1,143 total sales in a year

2.4 Data Quality Verification

Data Quality Verification is an important procedure in data management which guarantees the consistency, precision, dependability, and completeness of the data in a dataset. It involves assessing the dataset to make sure it satisfies predetermined standards and is appropriate for the purpose for which it is intended. Verifying the quality of data is essential in order to ensure accurate analysis, well-informed decision-making, and maintaining operational efficiency. The reason behind this is to prevent the occurrence of flawed insights and unreliable predictions.

2.4.1 Detect of Outliers

Measurement of central tendency and statistics				
	UnitPrice	Quantity	CustomerID	Sales
count	541909.000000	541909.000000	406829.000000	541909.000000
mean	4.611114	9.552250	15287.690570	17.987795
std	96.759853	218.081158	1713.600303	378.810824
min	-11062.060000	-80995.000000	12346.000000	-168469.600000
25%	1.250000	1.000000	13953.000000	3.400000
50%	2.080000	3.000000	15152.000000	9.750000
75%	4.130000	10.000000	16791.000000	17.400000
max	38970.000000	80995.000000	18287.000000	168469.600000

Figure 2.4.1.0

For data quality verification, we can use `data.describe()` in order to identify the outliers in the variable of the dataset such as the **Figure 2.4.1.0** shown above. For example, the minimum of the unit price and the quantity should not be lesser or equal to 0 in an online retail store dataset. Therefore, we can say that unit price and quantity variables contain outliers or inaccurate data. As sales is the multiplication of unit price and quantity, therefore if the unit price and quantity data is not cleaned, then the sales data will also consist of outliers. Besides that, we can also identify the missing values of customer ID as it has a lesser number of counts compared to other variables. We have also used a box plot in order to visualise and identify outliers which can be seen in **Figure 2.4.1.1** below.

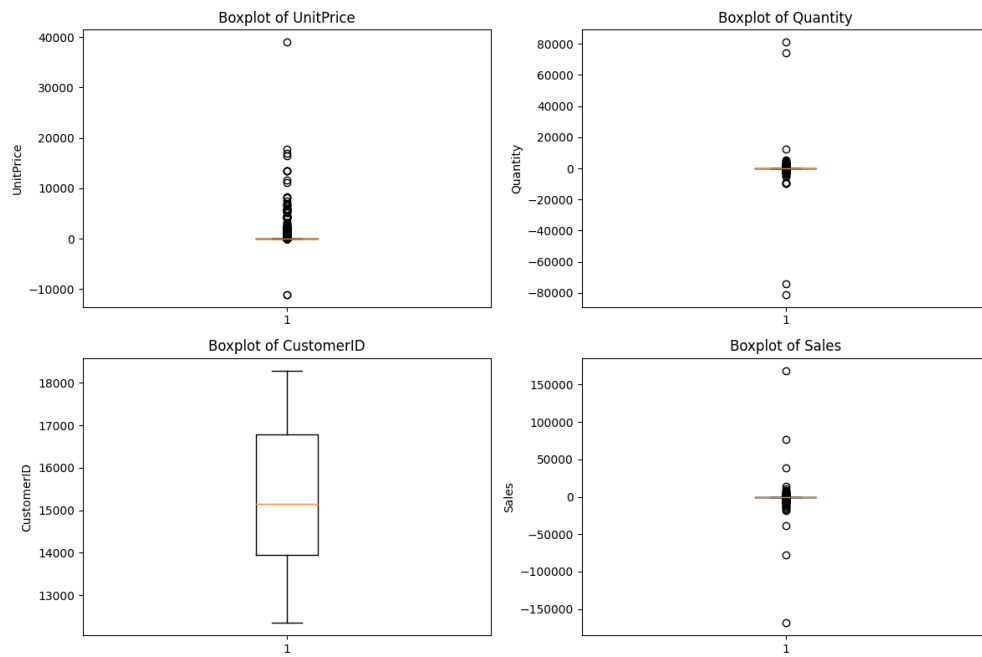


Figure 2.4.1.1

2.4.2 Verification of Data Types

	0
InvoiceNo	object
StockCode	object
Description	object
Quantity	int64
InvoiceDate	datetime64[ns]
UnitPrice	float64
CustomerID	float64
Country	object
Sales	float64
month	int32
year	int32
WeekDay	object
month_year	datetime64[ns]
Hour	int32

Figure 2.4.2.0

Verification of data types is also an important part in the data quality verification process to ensure the accuracy of calculations and operations that are performed. Wrongly identifying the data types may lead to incorrect results. By identifying the data types such as *Figure 2.4.2.0* above allow the analysts or data scientists to quickly identify the algorithms or tools that are able to be utilised in order to perform analysis which results in efficiency and effectiveness. Lastly, ensuring data in the correct format helps in maintaining data integrity and reliability of the result that has been interpreted using the data.

2.4.3 Detect of Missing Values

	InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice	CustomerID	Country	Sales	month	year	WeekDay	month_year	Hour
0	False	False	False	False	False	False	False	False	False	False	False	False	False	False
1	False	False	False	False	False	False	False	False	False	False	False	False	False	False
2	False	False	False	False	False	False	False	False	False	False	False	False	False	False
3	False	False	False	False	False	False	False	False	False	False	False	False	False	False
4	False	False	False	False	False	False	False	False	False	False	False	False	False	False
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
541904	False	False	False	False	False	False	False	False	False	False	False	False	False	False
541905	False	False	False	False	False	False	False	False	False	False	False	False	False	False
541906	False	False	False	False	False	False	False	False	False	False	False	False	False	False
541907	False	False	False	False	False	False	False	False	False	False	False	False	False	False
541908	False	False	False	False	False	False	False	False	False	False	False	False	False	False

541909 rows x 14 columns

Figure 2.4.3.0

0	
InvoiceNo	False
StockCode	False
Description	True
Quantity	False
InvoiceDate	False
UnitPrice	False
CustomerID	True
Country	False
Sales	False
month	False
year	False
WeekDay	False
month_year	False
Hour	False

Figure 2.4.3.1

0	
InvoiceNo	0
StockCode	0
Description	1454
Quantity	0
InvoiceDate	0
UnitPrice	0
CustomerID	135080
Country	0
Sales	0
month	0
year	0
WeekDay	0
month_year	0
Hour	0

Figure 2.4.3.2

In order to detect the missing values in the dataset that is given, we can use `data.isnull()` function to view and identify the missing values. However, if the dataset online retail is too large, it may be easy for us to overlook the null value using `data.isnull()`. Therefore, we have used `data.isnull.any()` to summaries and identified the null values. By observing **Figure 2.4.3.0**, **Figure 2.4.3.1**, **Figure 2.4.3.2** above, we can observe that there are missing values in description and customer ID as it returns true using the function mentioned above. Besides `data.isnull.any()` function, we can also use `data.isnull.sum()` function to sum up the total number of missing values for each of the variables in the dataset.

2.4.4 Detect of Duplicate Rows

Numbers of duplicates rows :5268													
InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice	CustomerID	Country	Sales	month	year	WeekDay	month_year	Hour
517	536409	21866	UNION JACK FLAG LUGGAGE TAG	1	2010-12-01 11:45:00	1.25	17908.0	United Kingdom	1.25	12	2010	Wednesday	2010-12-01 11
527	536409	22866	HAND WARMER SCOTTY DOG DESIGN	1	2010-12-01 11:45:00	2.10	17908.0	United Kingdom	2.10	12	2010	Wednesday	2010-12-01 11
537	536409	22900	SET 2 TEA TOWELS I LOVE LONDON	1	2010-12-01 11:45:00	2.95	17908.0	United Kingdom	2.95	12	2010	Wednesday	2010-12-01 11
539	536409	22111	SCOTTIE DOG HOT WATER BOTTLE	1	2010-12-01 11:45:00	4.95	17908.0	United Kingdom	4.95	12	2010	Wednesday	2010-12-01 11
555	536412	22327	ROUND SNACK BOXES SET OF 4 SKULLS	1	2010-12-01 11:49:00	2.95	17920.0	United Kingdom	2.95	12	2010	Wednesday	2010-12-01 11
...	...	...	...	...	...	...	...	...	...	...	...	...	...
541675	581538	22068	BLACK PIRATE TREASURE CHEST	1	2011-12-09 11:34:00	0.39	14446.0	United Kingdom	0.39	12	2011	Friday	2011-12-01 11
541689	581538	23318	BOX OF 6 MINI VINTAGE CRACKERS	1	2011-12-09 11:34:00	2.49	14446.0	United Kingdom	2.49	12	2011	Friday	2011-12-01 11
541692	581538	22992	REVOLVER WOODEN RULER	1	2011-12-09 11:34:00	1.95	14446.0	United Kingdom	1.95	12	2011	Friday	2011-12-01 11
541699	581538	22694	WICKER STAR	1	2011-12-09 11:34:00	2.10	14446.0	United Kingdom	2.10	12	2011	Friday	2011-12-01 11
541701	581538	23343	JUMBO BAG VINTAGE CHRISTMAS	1	2011-12-09 11:34:00	2.08	14446.0	United Kingdom	2.08	12	2011	Friday	2011-12-01 11
5268 rows x 14 columns													

Figure 2.4.4.0

In order to detect duplicate rows, the function duplicated() has been used in data quality verification in order to ensure there is no duplicate data in the dataset. The dataset, which is shown in **Figure 2.4.4.0**, seems to be duplicate, from invoice number, customer ID, invoice date, customer ID etc. However, they refer to the different stock code and description of product, which cannot be counted as duplicate data.

2.4.5 Verify and Validate Numeric Ranges

2.4.5.1 Validate using Quantity

Number of invalid numeric ranges according to quantity : 10627														
	InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice	CustomerID	Country	Sales	month	year	WeekDay	month_year	Hour
141	C536379	D	Discount	-1	2010-12-01 09:41:00	27.50	14527.0	United Kingdom	-27.50	12	2010	Wednesday	2010-12-01	9
154	C536383	35004C	SET OF 3 COLOURED FLYING DUCKS	-1	2010-12-01 09:49:00	4.65	15311.0	United Kingdom	-4.65	12	2010	Wednesday	2010-12-01	9
235	C536391	22556	PLASTERS IN TIN CIRCUS PARADE	-12	2010-12-01 10:24:00	1.65	17548.0	United Kingdom	-19.80	12	2010	Wednesday	2010-12-01	10
236	C536391	21984	PACK OF 12 PINK PAISLEY TISSUES	-24	2010-12-01 10:24:00	0.29	17548.0	United Kingdom	-6.96	12	2010	Wednesday	2010-12-01	10
237	C536391	21983	PACK OF 12 BLUE PAISLEY TISSUES	-24	2010-12-01 10:24:00	0.29	17548.0	United Kingdom	-6.96	12	2010	Wednesday	2010-12-01	10
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
540449	C581490	23144	ZINC T-LIGHT HOLDER STARS SMALL	-11	2011-12-09 09:57:00	0.83	14397.0	United Kingdom	-9.13	12	2011	Friday	2011-12-01	9
541541	C581499	M	Manual	-1	2011-12-09 10:28:00	224.69	15498.0	United Kingdom	-224.69	12	2011	Friday	2011-12-01	10
541715	C581568	21258	VICTORIAN SEWING BOX LARGE	-5	2011-12-09 11:57:00	10.95	15311.0	United Kingdom	-54.75	12	2011	Friday	2011-12-01	11
541716	C581569	84978	HANGING HEART JAR T-LIGHT HOLDER	-1	2011-12-09 11:58:00	1.25	17315.0	United Kingdom	-1.25	12	2011	Friday	2011-12-01	11
541717	C581569	20979	36 PENCILS TUBE RED RETROSPOT	-5	2011-12-09 11:58:00	1.25	17315.0	United Kingdom	-6.25	12	2011	Friday	2011-12-01	11
10627 rows x 14 columns														

Figure 2.4.5.1.0

Data quality verification also includes verifying and validating in numeric ranges such as the quantity. In an item purchased, there is a limitation where customers are not able to purchase products with the quantity more than 10,000. Besides that, according to **Figure 2.4.5.1.0**, a quantity that is lower or equal to zero is also not counted as a quantity of product that has been sold since that would affect sales being negative value therefore they can be deemed as outliers and it should be removed during the data cleaning process.

2.4.5.2 Validate using Unit Price

Number of invalid numeric ranges according to Unit Price: 2517															
	InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice	CustomerID	Country	Sales	month	year	WeekDay	month_year	Hour	
	622	536414	22139	NaN	56	2010-12-01 11:52:00	0.0	NaN	United Kingdom	0.0	12	2010	Wednesday	2010-12-01	11
	1970	536545	21134	NaN	1	2010-12-01 14:32:00	0.0	NaN	United Kingdom	0.0	12	2010	Wednesday	2010-12-01	14
	1971	536546	22145	NaN	1	2010-12-01 14:33:00	0.0	NaN	United Kingdom	0.0	12	2010	Wednesday	2010-12-01	14
	1972	536547	37509	NaN	1	2010-12-01 14:33:00	0.0	NaN	United Kingdom	0.0	12	2010	Wednesday	2010-12-01	14
	1987	536549	85226A	NaN	1	2010-12-01 14:34:00	0.0	NaN	United Kingdom	0.0	12	2010	Wednesday	2010-12-01	14
	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
	536981	581234	72817	NaN	27	2011-12-08 10:33:00	0.0	NaN	United Kingdom	0.0	12	2011	Thursday	2011-12-01	10
	538504	581406	46000M POLYESTER FILLER PAD 45x45cm	240	2011-12-08 13:58:00	0.0	NaN	United Kingdom	0.0	12	2011	Thursday	2011-12-01	13	
	538505	581406	46000S POLYESTER FILLER PAD 40x40cm	300	2011-12-08 13:58:00	0.0	NaN	United Kingdom	0.0	12	2011	Thursday	2011-12-01	13	
	538554	581408	85175	NaN	20	2011-12-08 14:06:00	0.0	NaN	United Kingdom	0.0	12	2011	Thursday	2011-12-01	14
	538919	581422	23169	smashed	-235	2011-12-08 15:24:00	0.0	NaN	United Kingdom	-0.0	12	2011	Thursday	2011-12-01	15
2517 rows x 14 columns															

Figure 2.4.5.2.0

Besides the quantity of the product sold, we should also verify and validate the unit price of items that has been sold should not be lower or equal to 0 which can be shown in **Figure 2.4.5.2.0**. For example, an item that is equal to zero may be a free gift for customers who purchase more than a certain amount which should not be included in the dataset to avoid inconsistency and inaccurate analysis.

2.4.5.3 Validate using Sales

Number of invalid numeric ranges according to Sales: 11805														
	InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice	CustomerID	Country	Sales	month	year	WeekDay	month_year	Hour
141	C536379	D	Discount	-1	2010-12-01 09:41:00	27.50	14527.0	United Kingdom	-27.50	12	2010	Wednesday	2010-12-01	9
154	C536383	35004C	SET OF 3 COLOURED FLYING DUCKS	-1	2010-12-01 09:49:00	4.65	15311.0	United Kingdom	-4.65	12	2010	Wednesday	2010-12-01	9
235	C536391	22556	PLASTERS IN TIN CIRCUS PARADE	-12	2010-12-01 10:24:00	1.65	17548.0	United Kingdom	-19.80	12	2010	Wednesday	2010-12-01	10
236	C536391	21984	PACK OF 12 PINK PAISLEY TISSUES	-24	2010-12-01 10:24:00	0.29	17548.0	United Kingdom	-6.96	12	2010	Wednesday	2010-12-01	10
237	C536391	21983	PACK OF 12 BLUE PAISLEY TISSUES	-24	2010-12-01 10:24:00	0.29	17548.0	United Kingdom	-6.96	12	2010	Wednesday	2010-12-01	10
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
540449	C581490	23144	ZINC T-LIGHT HOLDER STARS SMALL	-11	2011-12-09 09:57:00	0.83	14397.0	United Kingdom	-9.13	12	2011	Friday	2011-12-01	9
541541	C581499	M	Manual	-1	2011-12-09 10:28:00	224.69	15498.0	United Kingdom	-224.69	12	2011	Friday	2011-12-01	10
541715	C581568	21258	VICTORIAN SEWING BOX LARGE	-5	2011-12-09 11:57:00	10.95	15311.0	United Kingdom	-54.75	12	2011	Friday	2011-12-01	11
541716	C581569	84978	HANGING HEART JAR T-LIGHT HOLDER	-1	2011-12-09 11:58:00	1.25	17315.0	United Kingdom	-1.25	12	2011	Friday	2011-12-01	11
541717	C581569	20979	36 PENCILS TUBE RED RETROSPOT	-5	2011-12-09 11:58:00	1.25	17315.0	United Kingdom	-6.25	12	2011	Friday	2011-12-01	11
11805 rows x 14 columns														

Figure 2.4.5.3.0

The variable sales is a continuous numeric variable which should also be validated and verified in order to produce accurate and reliable analysis. In **Figure 2.4.5.3.0**, the total amount of sales should also not be equal or lower than zero. Data cleaning process is needed if there is some inconsistency or inaccuracy in the sales variable of the dataset.

3.0 Data Preparation

Data preparation is the first step in turning the available or existing data into a dataset that can be used during the modelling phase after we have understood and explored the data, which is usually normalised so that it can be used for data analysis models or machine learning systems to operate. Data preparation can be categorised into several components, which are the selection of data, data cleansing, data construction, data normalisation and data concatenation.

3.1 Data Selection

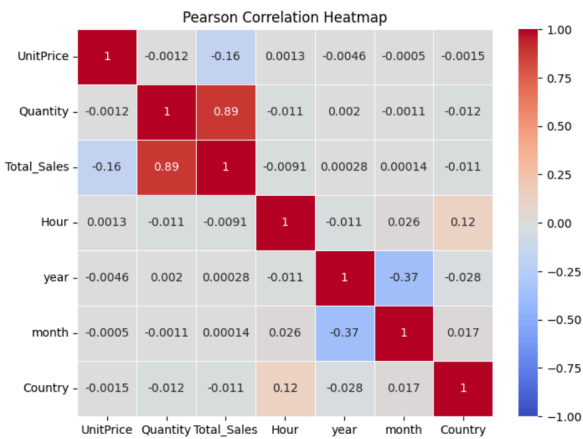


Figure 3.1.0

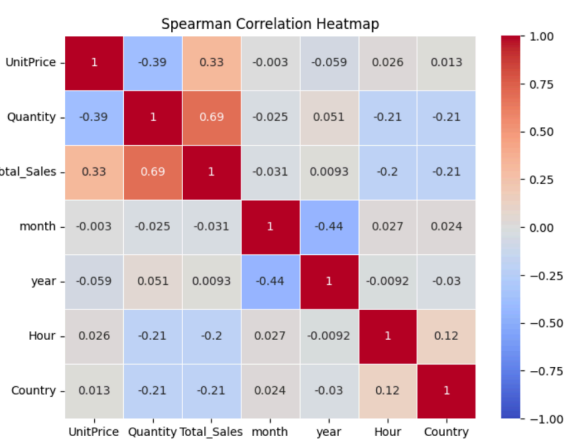


Figure 3.1.1

Data selection is a main step in data preparation where relevant data types, sources and tools for data collection are identified and chosen. For this case, the dataset used for analysis purposes is considered secondary data, which means the data was collected and potentially underwent processing by individuals. This distinction is very important as secondary data may have already proceeded with certain transformations which can influence its current form and usability. Data selection is crucial in data science because we need to select the right data that is aligned with the business objectives or goals of the analysis. This would help to ensure that the variables used in the analysis will be directly related to the issue at hand, which will increase the accuracy and usefulness of the insight gained from the analysis. Misleading conclusions can be avoided by reducing unnecessary information through selecting the right data.

As for the data selection process, we have used two methods in order to identify the relationships between the variables, which are the pearson correlation matrix and the spearman correlation matrix. The heatmap is used to clearly identify the relationship between variables with different colours which produces a clear effect for analysts to understand and observe the relationship well. Pearson correlation coefficient is used to measure the linear relationship between continuous variables by indicating how strongly two variables are related on a linear scale while the spearman relationship measures the rank-order relationship between variables, capturing both linear and nonlinear association.

3.1.1 Rationale of Inclusion

As we can tell from the correlation heatmap for pearson correlation coefficient and spearman correlation coefficient above, we can conclude that the quantity and sales variable has a strong relationship between each other in both the pearson correlation heatmap and the spearman correlation heatmap. Therefore, it is essential that the quantity variable is included in order to predict future sales accurately .

Since including only the quantity variable is not sufficient enough for the predictions. Other factors that may influence sales must also be considered. After discussing with the team, we concluded that variables such as unit price, country, month and hour should also be included in the data model due to the fact that these variables are likely to have significant impact on sales trends and patterns. The rationale behind the inclusion of these variables is based on each of their potential influences on sales. For example, unit price directly affects the total sales, while on the other hand, country may help in revealing geographic trends. Another example would be that seasonal trends or time-based sales variations can be captured through temporal context provided by the variables such as month, year and hour.

3.1.2 Summary of Selected Data

```
print(X.dtypes)
✓ 0.0s

Quantity      int64
UnitPrice     float64
month         int64
year          int64
Country       object
Description    object
dtype: object
```

Figure 3.1.2.0

Variables / Features : Quantity, Unit Price, Month, Year, Description and Country

Target : Total Sales

3.2 Data Cleaning

Data cleaning is a next process in data preparation which converts and refines data from a raw form into another format to ensure quality and usability of the data. Data cleaning also allows more convenient use in data analysing or exploring patterns and knowledge. It is usually a very time consuming part of the data preparations and requires a level of understanding of the data. The data cleaning process is crucial especially in getting rid of inconsistencies and noise values. It should be done in order to ensure the data reliability, consistency, accuracy and completeness of the dataset. Data can be missing due to several reasons such as non-applicable information, undisclosed information, error and outlier values.

There are several steps that can be taken involving filling in the missing values and removing corrupt or problematic data as well as to get rid of outliers such as incomplete data and inconsistency data. Data scientists can use the method replace, keep and delete in order to get rid of the data outliers. The missing value can be replaced with a known value, such as the mean, median and mode of the dataset. Besides that, we can also delete observations or variables for the data which may be not meaningful due to lots of missing values.

3.2.1 Deleting Values

3.2.1.1 Remove Quantity With Negative Values

Total of Quantity with negative values						Total Record after removing the negative values					
	InvoiceNo	StockCode	Description	Quantity	\		InvoiceNo	StockCode	Description	Quantity	\
141	C536379	D	Discount	-1		0	536365	85123A	WHITE HANGING HEART T-LIGHT HOLDER	6	
154	C536383	35004C	SET OF 3 COLOURED FLYING DUCKS	-1		1	536365	71053	WHITE METAL LANTERN	6	
235	C536391	22556	PLASTERS IN TIN CIRCUS PARADE	-12		2	536365	84406B	CREAM CUPID HEARTS COAT HANGER	8	
236	C536391	21984	PACK OF 12 PINK PAISLEY TISSUES	-24		3	536365	84029G	KNITTED UNION FLAG HOT WATER BOTTLE	6	
237	C536391	21983	PACK OF 12 BLUE PAISLEY TISSUES	-24		4	536365	84029E	RED WOOLLY HOTTIE WHITE HEART.	6	
...	...	...	...	...		...	...	...	...	...	
540449	C581490	23144	ZINC T-LIGHT HOLDER STARS SMALL	-11		541904	581587	22613	PACK OF 20 SPACEBOY NAPKINS	12	
541541	C581499	M	Manual	-1		541905	581587	22899	CHILDREN'S APRON DOLLY GIRL	6	
541715	C581568	21258	VICTORIAN SEWING BOX LARGE	-5		541906	581587	23254	CHILDRENS CUTLERY DOLLY GIRL	4	
541716	C581569	84978	HANGING HEART JAR T-LIGHT HOLDER	-1		541907	581587	23255	CHILDRENS CUTLERY CIRCUS PARADE	4	
541717	C581569	20979	36 PENCILS TUBE RED RETROSPOT	-5		541908	581587	22138	BAKING SET 9 PIECE RETROSPOT	3	
	InvoiceDate	UnitPrice	CustomerID	Country			InvoiceDate	UnitPrice	CustomerID	Country	
141	1/12/2010 9:41	27.50	14527.0	United Kingdom		0	1/12/2010 8:26	2.55	17850.0	United Kingdom	
154	1/12/2010 9:49	4.65	15311.0	United Kingdom		1	1/12/2010 8:26	3.39	17850.0	United Kingdom	
235	1/12/2010 10:24	1.65	17548.0	United Kingdom		2	1/12/2010 8:26	2.75	17850.0	United Kingdom	
236	1/12/2010 10:24	0.29	17548.0	United Kingdom		3	1/12/2010 8:26	3.39	17850.0	United Kingdom	
237	1/12/2010 10:24	0.29	17548.0	United Kingdom		4	1/12/2010 8:26	3.39	17850.0	United Kingdom	
...	...	...	...	...		...	...	...	...	...	
540449	9/12/2011 9:57	0.83	14397.0	United Kingdom		541904	9/12/2011 12:50	0.85	12680.0	France	
541541	9/12/2011 10:28	224.69	15498.0	United Kingdom		541905	9/12/2011 12:50	2.10	12680.0	France	
541715	9/12/2011 11:57	10.95	15311.0	United Kingdom		541906	9/12/2011 12:50	4.15	12680.0	France	
541716	9/12/2011 11:58	1.25	17315.0	United Kingdom		541907	9/12/2011 12:50	4.15	12680.0	France	
541717	9/12/2011 11:58	1.25	17315.0	United Kingdom		541908	9/12/2011 12:50	4.95	12680.0	France	
[10624 rows x 8 columns]						[531285 rows x 8 columns]					

Table 3.2.1.1.0

The reason that we need to clean the unit price negative values is also the same as the quantity. Based on **Table 3.2.1.1.0**, most of the negative values in unit price are customers who had cancelled their orders. The unit price should be more than 0 because that there is no price is 0 except that it is free. Inside our dataset, there are 10,624 rows of data that handle the negative value. The reason that the dataset handles the negative value is because all the invoice No. of these negative values starts with a 'C'. It means that all these data are cancelled by the customer. So after we remove the unit price that handles the negative value, the new total will be 531,285 data.

3.2.1.2 Remove Unit Price With Negative Values

Total of Unit Price with negative values						Total Record after removing the negative values					
<pre> InvoiceNo StockCode Description Quantity InvoiceDate \ 299983 A563186 B Adjust bad debt 1 12/8/2011 14:51 299984 A563187 B Adjust bad debt 1 12/8/2011 14:52 UnitPrice CustomerID Country 299983 -11062.06 NaN United Kingdom 299984 -11062.06 NaN United Kingdom </pre>						<pre> InvoiceNo StockCode Description Quantity \ 0 536365 85123A WHITE HANGING HEART T-LIGHT HOLDER 6 1 536365 71053 WHITE METAL LANTERN 6 2 536365 844068 CREAM CUPID HEARTS COAT HANGER 8 3 536365 84029G KNITTED UNION FLAG HOT WATER BOTTLE 6 4 536365 84029E RED WOOLLY HOTTIE WHITE HEART. 6 ... 541904 581587 22613 PACK OF 20 SPACEBOY NAPKINS 12 541905 581587 22899 CHILDREN'S APRON DOLLY GIRL 6 541906 581587 23254 CHILDRENS CUTLERY DOLLY GIRL 4 541907 581587 23255 CHILDRENS CUTLERY CIRCUS PARADE 4 541908 581587 22138 BAKING SET 9 PIECE RETROSPOT 3 InvoiceDate UnitPrice CustomerID Country 0 1/12/2010 8:26 2.55 17850.0 United Kingdom 1 1/12/2010 8:26 3.39 17850.0 United Kingdom 2 1/12/2010 8:26 2.75 17850.0 United Kingdom 3 1/12/2010 8:26 3.39 17850.0 United Kingdom 4 1/12/2010 8:26 3.39 17850.0 United Kingdom ... 541904 9/12/2011 12:50 0.85 12680.0 France 541905 9/12/2011 12:50 2.10 12680.0 France 541906 9/12/2011 12:50 4.15 12680.0 France 541907 9/12/2011 12:50 4.15 12680.0 France 541908 9/12/2011 12:50 4.95 12680.0 France [531283 rows x 8 columns] </pre>					

Table 3.2.1.2.0

The reason that we need to remove the quantity values is negative because in the online retails datasets, the negative values in unit price often do not comply with the logic. But based on **Table 3.2.1.2.0**, the negative value means that the customer cancelled the order. These negative values are not viable for the analysing process. In our dataset, there are a total of 2 records with negative values. After removing records that have unit price with a negative value, we have a total of 531,283 data.

3.2.1.3 Remove StockCode with values of Dot and Post

Total of StockCode with Dot and Post values						Total Record after removing Dot and Post values					
InvoiceNo	StockCode	Description	Quantity	InvoiceDate	\	InvoiceNo	StockCode	Description	Quantity	\	
45	536370	POST	POSTAGE	3	1/12/2010 8:45	0	536365	85123A	WHITE HANGING HEART T-LIGHT HOLDER	6	
377	536403	POST	POSTAGE	1	1/12/2010 11:27	1	536365	71053	WHITE METAL LANTERN	6	
1113	536527	POST	POSTAGE	1	1/12/2010 13:04	2	536365	84406B	CREAM CUPID HEARTS COAT HANGER	8	
1802	536544	DOT	DOTCOM POSTAGE	1	1/12/2010 14:32	3	536365	84029G	KNITTED UNION FLAG HOT WATER BOTTLE	6	
3014	536592	DOT	DOTCOM POSTAGE	1	1/12/2010 17:06	4	536365	84029E	RED WOOLLY HOTTIE WHITE HEART.	6	
...	...	...	...	...	...	...	...	...	...	...	
530594	581494	POST	POSTAGE	2	9/12/2011 10:13	529439	581587	22613	PACK OF 20 SPACEBOY NAPKINS	12	
530918	581498	DOT	DOTCOM POSTAGE	1	9/12/2011 10:26	529440	581587	22899	CHILDREN'S APRON DOLLY GIRL	6	
531104	581570	POST	POSTAGE	1	9/12/2011 11:59	529441	581587	23254	CHILDRENS CUTLERY DOLLY GIRL	4	
531141	581574	POST	POSTAGE	2	9/12/2011 12:09	529442	581587	23255	CHILDRENS CUTLERY CIRCUS PARADE	4	
531142	581578	POST	POSTAGE	3	9/12/2011 12:16	529443	581587	22138	BAKING SET 9 PIECE RETROSPOT	3	
UnitPrice	CustomerID	Country				InvoiceDate	UnitPrice	CustomerID	Country		
45	18.00	12583.0	France			0	1/12/2010 8:26	2.55	17850.0	United Kingdom	
377	15.00	12791.0	Netherlands			1	1/12/2010 8:26	3.39	17850.0	United Kingdom	
1113	18.00	12662.0	Germany			2	1/12/2010 8:26	2.75	17850.0	United Kingdom	
1802	569.77	NaN	United Kingdom			3	1/12/2010 8:26	3.39	17850.0	United Kingdom	
3014	607.49	NaN	United Kingdom			4	1/12/2010 8:26	3.39	17850.0	United Kingdom	
...	...	...	...			...	...	...	...	...	
530594	18.00	12518.0	Germany			529439	9/12/2011 12:50	0.85	12680.0	France	
530918	1714.17	NaN	United Kingdom			529440	9/12/2011 12:50	2.10	12680.0	France	
531104	18.00	12662.0	Germany			529441	9/12/2011 12:50	4.15	12680.0	France	
531141	18.00	12526.0	Germany			529442	9/12/2011 12:50	4.15	12680.0	France	
531142	18.00	12713.0	Germany			529443	9/12/2011 12:50	4.95	12680.0	France	
[1839 rows x 8 columns]						[529444 rows x 8 columns]					

Table 3.2.1.3.0

Based on **Table 3.2.1.3.0**, there are a total of 1839 records with the StockCode having the value of Dot and Post which should make any sense and it will affect the outcome of the analysis. Removing them is necessary. A total of 529,444 records left after we have removed them from the dataset.

3.2.1.4 Remove Description with Null Value

Total of Description with null value

InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice	\
611	536414	22139	NaN	56	1/12/2010 11:52	0.0
1954	536545	21134	NaN	1	1/12/2010 14:32	0.0
1955	536546	22145	NaN	1	1/12/2010 14:33	0.0
1956	536547	37509	NaN	1	1/12/2010 14:33	0.0
1957	536549	85226A	NaN	1	1/12/2010 14:34	0.0
...	...	...	...	...	...	...
521379	581103	22689	NaN	4	7/12/2011 11:58	0.0
522959	581203	23406	NaN	15	7/12/2011 18:31	0.0
522960	581209	21620	NaN	6	7/12/2011 18:35	0.0
524586	581234	72817	NaN	27	8/12/2011 10:33	0.0
526137	581408	85175	NaN	20	8/12/2011 14:06	0.0
CustomerID Country						
611	NaN	United Kingdom				
1954	NaN	United Kingdom				
1955	NaN	United Kingdom				
1956	NaN	United Kingdom				
1957	NaN	United Kingdom				
...	...	...				
521379	NaN	United Kingdom				
522959	NaN	United Kingdom				
522960	NaN	United Kingdom				
524586	NaN	United Kingdom				
526137	NaN	United Kingdom				
[587 rows x 8 columns]						
Number of rows with empty or whitespace-only Description: 587						

Total Record after removing null value records

InvoiceNo	StockCode	Description	Quantity	\		
0	536365	85123A	WHITE HANGING HEART T-LIGHT HOLDER	6		
1	536365	71053	WHITE METAL LANTERN	6		
2	536365	84406B	CREAM CUPID HEARTS COAT HANGER	8		
3	536365	84029G	KNITTED UNION FLAG HOT WATER BOTTLE	6		
4	536365	84029E	RED WOOLLY HOTTIE WHITE HEART.	6		
...	...	...	...	...		
529439	581587	22613	PACK OF 20 SPACEBOY NAPKINS	12		
529440	581587	22899	CHILDREN'S APRON DOLLY GIRL	6		
529441	581587	23254	CHILDRENS CUTLERY DOLLY GIRL	4		
529442	581587	23255	CHILDRENS CUTLERY CIRCUS PARADE	4		
529443	581587	22138	BAKING SET 9 PIECE RETROSPOT	3		
InvoiceDate UnitPrice CustomerID Country \						
0	2010-12-01 08:26:00	2.55	17850.0	United Kingdom		
1	2010-12-01 08:26:00	3.39	17850.0	United Kingdom		
2	2010-12-01 08:26:00	2.75	17850.0	United Kingdom		
3	2010-12-01 08:26:00	3.39	17850.0	United Kingdom		
4	2010-12-01 08:26:00	3.39	17850.0	United Kingdom		
...	...	...	...	...		
529439	2011-12-09 12:50:00	0.85	12680.0	France		
529440	2011-12-09 12:50:00	2.10	12680.0	France		
529441	2011-12-09 12:50:00	4.15	12680.0	France		
529442	2011-12-09 12:50:00	4.15	12680.0	France		
529443	2011-12-09 12:50:00	4.95	12680.0	France		
Total_Sales month year WeekDay month_year Hour						
0	15.30	12	2010	Wednesday	2010-12-01	8
1	20.34	12	2010	Wednesday	2010-12-01	8
2	22.00	12	2010	Wednesday	2010-12-01	8
3	20.34	12	2010	Wednesday	2010-12-01	8
4	20.34	12	2010	Wednesday	2010-12-01	8
...	...	...	...	...	...	...
529439	10.20	12	2011	Friday	2011-12-01	12
529440	12.60	12	2011	Friday	2011-12-01	12
529441	16.60	12	2011	Friday	2011-12-01	12
529442	16.60	12	2011	Friday	2011-12-01	12
529443	14.85	12	2011	Friday	2011-12-01	12
[528857 rows x 14 columns]						
Number of rows: 528857						

Table 3.2.1.4.0

Based on **Table 3.2.1.4.0**, we found that there are a total of 587 records of data that the Description is empty. The reason that we remove the data description is because if it is empty it may lead to data imbalance or bias in analysis. So, removing these records can help maintain data consistency and integrity. After we remove the record that has a null value in the description variable, we have a total of 528,857 records left.

3.2.1.5 Remove the outlier

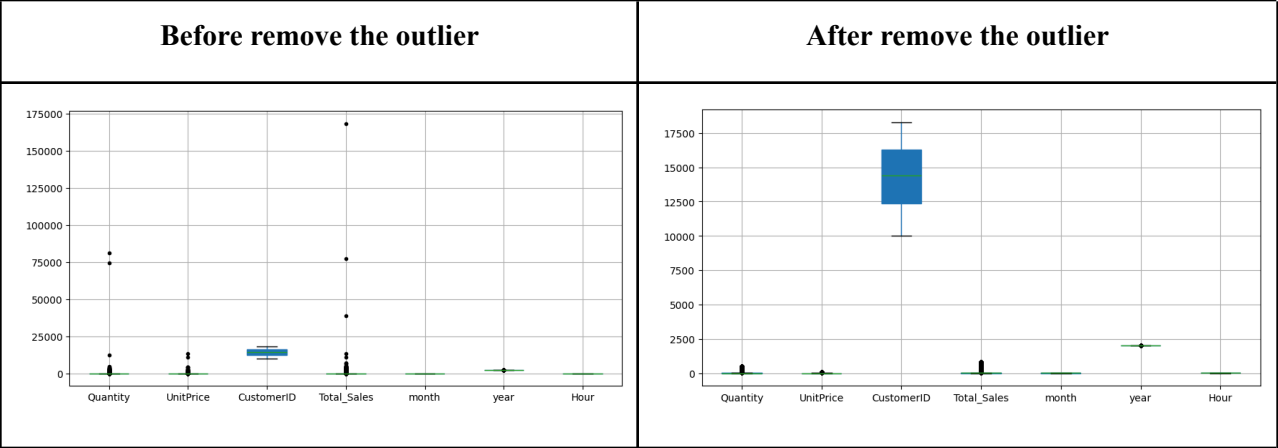


Table 3.2.1.5.0.

When we calculate that all the z-score is larger than 3, it will show the outliers. So we need to remove the outliers based on *Table 3.2.1.5.0* with the given box plot layout.

3.2.1 Randomly Add New Customer ID Based On Invoice No

Table 3.2.1.0

51

3.3 Data Construction

Data construction is a data preprocessing process of creating and preparing datasets for analysis and modelling, including data collection, cleaning, transformation, integration, and structuring to ensure that it is in a usable format for machine learning and data analysis tasks. It is quite important in data preparation as it is able to improve the data quality. A proper data construction is able to ensure that the data is accurate, consistent, relevant, and reliable for data analysis and modelling in order to gain useful insights or patterns. Besides that, data construction can facilitate the analysis, as well as structured data will make it easier to apply to various analytical techniques. A well-constructed dataset allows the model to train accurately and generalise because it is able to minimise the risk of errors and biases that can arise from poorly prepared data, leading to more valid results.

In data construction, based on *Figure 3.3.0*, *Figure 3.3.1* and *Figure 3.3.2*, we have differentiated those variables to categorical and continuous in order to provide ease in analysing the data during the modelling phases.

```
#Define that which is categorical and continuous
categorical_col = ['InvoiceNo','StockCode','Description','Country']
continuous_col = ['Quantity','UnitPrice','Total_Sales','month','year','Hour']

categorical_df = df[categorical_col]
continuous_df = df[continuous_col]
```

Figure 3.3.0

	InvoiceNo	StockCode	Description	Country
0	536365	85123A	WHITE HANGING HEART T-LIGHT HOLDER	United Kingdom
1	536365	71053	WHITE METAL LANTERN	United Kingdom
2	536365	84406B	CREAM CUPID HEARTS COAT HANGER	United Kingdom
3	536365	84029G	KNITTED UNION FLAG HOT WATER BOTTLE	United Kingdom
4	536365	84029E	RED WOOLLY HOTTIE WHITE HEART.	United Kingdom
...	...	...	...	...
528852	581587	22613	PACK OF 20 SPACEBOY NAPKINS	France
528853	581587	22899	CHILDREN'S APRON DOLLY GIRL	France
528854	581587	23254	CHILDRENS CUTLERY DOLLY GIRL	France
528855	581587	23255	CHILDRENS CUTLERY CIRCUS PARADE	France
528856	581587	22138	BAKING SET 9 PIECE RETROSPOT	France

Figure 3.3.0

	Quantity	UnitPrice	Total_Sales	month	year	Hour
0	6	2.55	15.30	12	2010	8
1	6	3.39	20.34	12	2010	8
2	8	2.75	22.00	12	2010	8
3	6	3.39	20.34	12	2010	8
4	6	3.39	20.34	12	2010	8
...	...	...	...	...	...	...
528852	12	0.85	10.20	12	2011	12
528853	6	2.10	12.60	12	2011	12
528854	4	4.15	16.60	12	2011	12
528855	4	4.15	16.60	12	2011	12
528856	3	4.95	14.85	12	2011	12

Figure 3.3.0

3.4 Data Normalisation

Data normalisation is a process of scaling numerical features to a specific range, typically between 0 and 1. This process is carried out in order to ensure that different variables with different ranges are comparable and will not disproportionately influence the model. For this case, we will be applying the MinMax Normalisation with the formula shown in *Figure 3.4.0*.

$$X_{new} = \frac{X_{old} - \min(X_{old})}{\max(X_{old}) - \min(X_{old})}(\text{newmax} - \text{newmin}) + \text{newmin},$$

whereby newmax and newmin are the newly imposed maximum and minimum (e.g., 1 and 0).

Figure 3.4.0

Based on *Figure 3.4.1* and *Figure 3.4.2*, We will apply the MinMaxScaler to the continuous columns of the dataset. For checking purposes, we have printed the categorical and continuous columns separately. After the normalisation process, the DataFrame is reset and combined to allow clean and consistent structure

```
from sklearn.preprocessing import MinMaxScaler
scaler = MinMaxScaler()
normalized_continuous_df = pd.DataFrame(scaler.fit_transform(continuous_df), columns=continuous_col)

#Just for checking#

print(len(categorical_df))
print(len(normalized_continuous_df))

# Check indices
print(categorical_df.index)
print(normalized_continuous_df.index)

print(categorical_df.columns)
print(normalized_continuous_df.columns)

categorical_df = categorical_df.reset_index(drop=True)
normalized_continuous_df = normalized_continuous_df.reset_index(drop=True)
```

Figure 3.4.1

```
528857
528857
RangeIndex(start=0, stop=528857, step=1)
RangeIndex(start=0, stop=528857, step=1)
Index(['InvoiceNo', 'StockCode', 'Description', 'Country'], dtype='object')
Index(['Quantity', 'UnitPrice', 'Total_Sales', 'month', 'year', 'Hour'], dtype='object')
```

Figure 3.4.2

The model will be better equipped in handling variations in scale between different features in order to ensure balanced learning.

3.5 Data Concatenation

In this phase, we will be combining multiple data frames into a single unified data frame before moving on to the modelling process. Based on **Figure 3.5.0** and **Figure 3.5.1**, for each categorical column, the data type was changed to category. After that, the categorical data frame will merge with the normalised continuous data frame into a single data frame. The columns from the normalised continuous data frame will be added to the end of the categorical data frame.

```
for col in categorical_col:
    categorical_df[col] = categorical_df[col].astype('category')

combine_df = pd.concat([categorical_df, normalized_continuous_df], axis=1)

#Result
print(combine_df.head())
```

Figure 3.5.0

	InvoiceNo	StockCode	Description	Country	\	
0	536365	85123A	WHITE HANGING HEART T-LIGHT HOLDER	United Kingdom		
1	536365	71053	WHITE METAL LANTERN	United Kingdom		
2	536365	84406B	CREAM CUPID HEARTS COAT HANGER	United Kingdom		
3	536365	84029G	KNITTED UNION FLAG HOT WATER BOTTLE	United Kingdom		
4	536365	84029E	RED WOOLLY HOTTIE WHITE HEART.	United Kingdom		
	Quantity	UnitPrice	Total_Sales	month	year	Hour
0	0.000062	0.000188	0.000091	1.0	0.0	0.142857
1	0.000062	0.000250	0.000121	1.0	0.0	0.142857
2	0.000086	0.000203	0.000131	1.0	0.0	0.142857
3	0.000062	0.000250	0.000121	1.0	0.0	0.142857
4	0.000062	0.000250	0.000121	1.0	0.0	0.142857

Figure 3.5.1

4.0 Modelling

In data science, modelling is a method of developing mathematical models that can analyse trends, generate predictions, or assist in decision-making using data and algorithms. This involves selecting the right algorithms, using data to train models, and assessing how well they work to address certain issues or provide answers.

Modelling is essential to converting unstructured, raw data into useful, organised insights that support operational effectiveness and strategic decision-making. Fundamentally, modelling is building computational or mathematical frameworks that analyse data, spot trends, and interpret information. Modelling can be used in numerous corporate operations, including sales forecasting, supply chain optimization, fraud detection, and customer experience personalization.

Despite establishing estimations, modelling is vital because it improves in the understanding and measurement of complicated relationships that exist within an organisation's data. For instance, by utilising the regression models, it allows businesses to determine how different factors affect sales or consumer behaviour.

Effective modelling also supports hypothesis testing and validation by providing evidence-based insights. It allows organisations to test different scenarios, understand potential outcomes, and make data-driven decisions with greater confidence. Furthermore, models can be used to optimise resource allocation and operational processes, leading to cost savings and efficiency gains.

In a competitive landscape, modelling enables organisations to remain agile and responsive to market changes. By leveraging data-driven insights, businesses can anticipate trends, adapt strategies, and innovate more effectively. Overall, modelling is a cornerstone of data science that enhances decision-making, drives strategic planning, and ultimately contributes to achieving a sustainable competitive advantage.

4.1 Train Test Split

The "train test split" method divides the datasets into the test set and the training set, two subsets that are typically utilised in supervised learning. While the training set is used to train the model, the test set is used to evaluate how well the completed model performs. Splitting the train and test is a crucial method for efficiently preventing overfitting of the model. You can make sure that the model gets tested on data that hasn't been seen by setting aside some of the data for testing. This guarantees that the model generalises well to fresh, unknown data and assists in the detection of overfitting.

```
from sklearn.model_selection import train_test_split

X = dfNew[['Quantity', 'UnitPrice', 'month', 'year', 'Hour']] # Features
y = dfNew['Total_Sales'] # Target variable

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

print("Training data shape:", X_train.shape)
print("Testing data shape:", X_test.shape)
print("Training labels shape:", y_train.shape)
print("Testing labels shape:", y_test.shape)
```

Figure 4.1.0

```
Training data shape: (423085, 5)
Testing data shape: (105772, 5)
Training labels shape: (423085,)
Testing labels shape: (105772,)
```

Figure 4.1.1

Based on **Figure 4.1.0**, 20% of the data will be used for testing while 80% will be used for training. **Figure 4.1.1** shows the result of the split.

4.2 Evaluation Metrics

Evaluation metrics are quantifiable measurements used to evaluate the performance of a data science model. They offer insights into a model's advantages and disadvantages as well as performance metrics. These metrics are crucial for comparing various models or algorithms as well as for comprehending how well a model generalises to new, unseen data. Supervised and unsupervised learning involve different tasks and yield different types of output, evaluation measures for each have diverse applications. Evaluation measures for supervised learning, such as regression metrics, are categorised into different categories. We have also use several evaluation metrics in order to evaluate our model, such as the Mean Square Error, Mean Absolute Percentage Error, Coefficient of Determination, Root Mean Square Error and Mean Absolute Error

4.2.1 Evaluation Metrics for Supervised Learning

4.2.1.0 Mean Square Error (MSE)

The Mean Square Error metric is assessed commonly in order to evaluate regression model performance. It calculates the average squared difference between the expected and actual numbers, or the average of the squares of the errors.

Formula:

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Figure 4.2.1.0.0

- n : the number of observations.
- y_i : the actual value for the i -th observation.
- $\hat{y}_i$: the predicted value for the i -th observation.

Mean Squared Error is a critical statistic for evaluating the precision of regression models, which measures the degree to which a model's predictions agree with actual data. As a result of amplifying larger errors and emphasising more significant disparities, it computes the average of the squared differences between the predicted and actual values. It encourages the model to minimise these bigger deviations, MSE is especially useful in situations where significant errors are expensive or unwanted. MSE furthermore offers a standardised metric that makes it easier to compare various models or algorithms and makes it possible to assess their relative performance in a clear and concise manner. Model selection and overall model optimization are facilitated by MSE, which provides a common benchmark to help choose the best model or configuration effectively and efficiently.

There are various constraints associated with Mean Squared Error (MSE) that can influence its value as a performance statistic. First of all, because MSE squares the errors and assigns excessive weight to extreme values, it is extremely susceptible to outliers. Due to its sensitivity, a few large errors may have a disproportionately large impact on the overall MSE, which could distort the evaluation of the model's performance and produce false results. Second, MSE is not easily interpreted because it is given in response to variable squared units, which makes it challenging to practically apply to the real data. Finally, when

comparing models between datasets or problems with different scales, it can be challenging because MSE does not normalise errors based on the data's scale. This absence of acceptance

4.2.1.1 Mean Absolute Percentage Error (MAPE)

MAPE(Mean Absolute Percentage Error) is used as an indicator in supervised learning to evaluate the prediction performance and assess the accuracy of a forecasting model such as the regression models. It measures the palate between model predictions and actual values and expresses it as a percentage for understanding and comparison.

Formula:

$$\text{MAPE} = \frac{1}{n} \sum_{i=1}^n \left| \frac{A_i - F_i}{A_i} \right| \times 100\%$$

Figure 4.2.1.1.0

- A_i : actual value
- F_i : forecast value
- n : total number of observations

Mean Absolute Percentage Error provides error message in percentage terms, which make it directly and straightforward make the ease of communication and understand the model's accuracy for professions and non-professions. If the value of MAPE is 8%, therefore we can say that the forecast deviates from the actuals values by 8%. Mean Absolute Percentage Error also gives a normalised measure of accuracy, allowing effective and efficient comparisons between different models. This is especially useful in selecting the best model among various alternatives, as it standardises the error measure regardless of the scale of the data. Mean Absolute Percentage Error are used to help in evaluating reliability of forecasts models, thereby facilitating more information decision making and enhancing decision support in a business entity.

However, Mean Absolute Percentage Error might hit a limitation when actual values are zero or very nearly zero, there may be issues. This is due to the possibility of ambiguous or excessively large percentage errors. Due to this restriction, it is less appropriate for datasets that frequently contain zero values or those that are crucial. In addition, MAPE handles under- and over-forecasting equally, which may not always have the desired practical effects. In certain situations, over- or under-forecasting may have different costs or effects, and MAPE does not account for this asymmetry. In addition to being sensitive to small values, the Mean Absolute Percentage Error can also disproportionately affect MAPE, which could skew the results. When

there is a significant degree of data variability, it may be necessary to use several measures or make changes in order to provide a thorough assessment of forecast accuracy.

4.2.1.2 R-Squared (Coefficient of Determination)

R-Squared (R^2), as known as the Coefficient of Determination, is a statistical measure, quantifies the proportion of variance in the dependent variable explained by the independent variables in the regression model. It is used to measure how well a model fits data, and how well it can predict future outcomes. The larger the R^2 value means that it is better fit to the model.

Formula :

$$R^2 = 1 - \frac{RSS}{TSS}$$

Figure 4.2.1.2.0

- R^2 : coefficient of determination
- RSS : sum of squares of residuals
- TSS : total sum of squares

The regression model's fit to the data is evaluated using R-squared. A higher R-squared value suggests a better fit since it shows that the model explains a greater percentage of the variance in the dependent variable. It additionally permits the comparison of various models run on the same dataset. Presuming other parameters such as model complexity are equivalent, models with greater R-squared values are typically favoured. In addition, R-squared is used to comprehend the correlations and offers information on the proportion of the dependent variable's variability that can be accounted for by the independent variables. This facilitates comprehension of the model's linkages' importance and strength. Additionally, R-squared can be used as a performance standard for models. It assists in determining if model modifications increase the model's explanatory power and in analysing the effects of adding or removing predictors.

Although a high R-squared value suggests a strong fit, it is not sufficient to demonstrate that the independent and dependent variables are causally related. Causation cannot be established by correlation alone. Besides that, Overfitting may result in a high R-squared, particularly with models with a large number of predictors. When a model overfits, it fails to identify or detect underlying patterns in the training data and instead identifies noise. This results in high R-squared values but a part of a part performance on fresh data. When it comes to non-linear models or models with non-linear variable relationships, R-squared provides less information. It might not accurately represent how models with intricate interactions behave. Whether or not

the additional predictors in a model are significant, the R-squared will always rise or remain constant when more predictors are added. As a result, R-squared may be less helpful in determining the actual value of other predictors. If there are outliers in the data, such as non-constant variance, R-squared may not be accurate. Its reliability may be affected by departures from the assumption that mistakes will vary continuously.

4.2.1.3 Root Mean Square Error

Root Mean Square Error (RMSE) is a type of measurement to measure the accuracy of a predictive model by quantifying the difference between the actual values and the values predicted by the model. It represents the square root of the average squared differences between the actual values and the forecasted values.

RMSE provides an indication of how well the model's predictions match the actual data.

Formula :

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (A_i - F_i)^2}$$

Figure 4.2.1.3.0

- n : the number of observations.
- A_i : the actual value for the i -th observation.
- F_i : the forecasted value for the i -th observation.
- $(A_i - F_i)^2$: the squared difference between the actual value and the forecasted

Since the Root Mean Square Error is expressed in the same units as the data, it is easier to understand and analyse. It gives a clear indication of the extent of mistakes. RMSE aids in assessing a predictive model's effectiveness. Better model performance is suggested by a lower RMSE, which shows that the model's predictions are closer to the actual values. When comparing different models, RMSE is helpful. In general, models with lower RMSE values are favoured since they show fewer prediction mistakes. Because of the squaring of errors, RMSE is very sensitive to big mistakes and can reveal significant differences between actual and anticipated values.

Since mistakes are squared, RMSE assigns disproportionate weight to large errors. Because of this, it may be unduly sensitive to outliers, which could distort the evaluation of the model's overall performance. The root mean square error (RMSE) is dependent on the data scale and is not standardised. This implies that RMSE values between models or datasets with varying units are not directly comparable. Small changes in RMSE may be hard to discern in very large datasets, and RMSE may not accurately represent the performance of the model. While RMSE quantifies the size of errors, it does not reveal whether the model consistently over- or underpredicts, information that could be crucial in assessing model performance.

4.2.1.4 Mean Absolute Error (MAE)

Mean Absolute Error (MAE) is a common metric used to evaluate the accuracy of a predictive model. It measures the average magnitude of errors in a set of predictions, without considering their direction. Unlike Root Mean Square Error (RMSE), MAE provides a straightforward interpretation as it averages the absolute differences between actual and predicted values. Mean Absolute Error (MAE) is defined as the average of the absolute differences between the actual values and the forecasted values. It provides a clear indication of how far off predictions are from the actual values, with lower MAE values indicating better model performance.

Formula :

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |A_i - F_i|$$

Figure 4.2.1.4.0

- n is the number of observations.
- A_i is the actual value for the i-th observation.
- F_i is the forecasted value for the i-th observation.
- $|A_i - F_i|$ is the absolute difference between the actual and forecasted values.

The Mean Absolute Error, which is the average of the absolute differences between the actual and expected values, offers a straightforward and understandable indicator of prediction accuracy. Since the metric and the data are expressed in the same units, it is easy to evaluate and comprehend the size of mistakes in relation to the original data. Since the squaring of differences causes Root Mean Square Error (RMSE) to be sensitive to big errors, Mean Absolute Error addresses all errors equally by taking their absolute values into account. Because of this feature, the MAE offers a more robust and balanced perspective of prediction accuracy and is less susceptible to outliers and extreme values. The Mean Absolute Error formula's ease of computation is ensured by its simplicity, which requires only fundamental arithmetic operations. Because of its simplicity of computation, MAE is a useful option for many applications, particularly those requiring a direct and uncomplicated evaluation of prediction accuracy. Mean Absolute Error is also very helpful when comparing the accuracy of various forecasting models. Better overall predictive performance is indicated by models with lower Mean Absolute Error values, which show that the model's predictions are, on average, closer to the actual values.

Mean Absolute Error has certain restrictions. The fact that MAE does not take into consideration the magnitude of mistakes larger than their absolute size is one of its main limitations. Because of this, big errors are given the same weight as tiny errors, which occasionally masks how serious prediction errors are. For some analyses or applications where the cost of large errors is disproportionately high, MAE may not adequately represent the impact of very substantial large deviations. Furthermore, the Mean Absolute Error is not sensitive to the size of errors that are larger than their absolute magnitude because it is computed using the absolute values of errors. Due to its inability to distinguish between the impacts of little and large errors, MAE may not be as useful in situations where knowing the extent of errors is crucial. For instance, MAE by itself could not give a complete picture of model performance in circumstances where big errors might have more detrimental effects. MAE's dependence on data scale and lack of normalisation are more drawbacks. This means that when comparing datasets or models with varying scales or units, MAE values cannot be directly compared. Consequently, when comparing across various contexts or datasets, it might be difficult to evaluate model performance in a uniform manner.

4.4 Linear Regression Algorithm

4.4.1 Algorithms and Assumptions

Algorithms

Linear Regression is a statistical model that calculates the linear connection between a scalar response and one or more explanatory variables. Simple linear regression is used when there is only one explanatory variable while multiple linear regression is used when there are numerous explanatory variables.

Assumptions of Linear Regression Algorithms

1. The relationship between the dependent and independent variables is linear.
2. The residuals (errors) are independent.
3. The residuals have constant variance.
4. The residuals are normally distributed.

4.4.2 Model Building

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_absolute_percentage_error, mean_squared_error, r2_score
from math import sqrt

# Features and target variable
X = dfNew[['Quantity', 'UnitPrice', 'month', 'year', 'Country', 'Description']] # Features
y = dfNew['Total_Sales'] # Target variable

# Define the preprocessor with both scaling and encoding
preprocessor = ColumnTransformer(
    transformers=[
        ('num', StandardScaler(), ['Quantity', 'UnitPrice', 'month', 'year']),
        ('cat', OneHotEncoder(handle_unknown='ignore'), ['Country', 'Description'])
    ])

# Create the pipeline with preprocessing and model
pipeline = Pipeline(steps=[
    ('preprocessor', preprocessor),
    ('model', LinearRegression())
])

# Split the data into training and testing datasets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Fit the pipeline with the training data
pipeline.fit(X_train, y_train)

# Make predictions with the test data
y_pred = pipeline.predict(X_test)
```

```
# Evaluate the model
mape = mean_absolute_percentage_error(y_test, y_pred)
mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
rmse = sqrt(mse)
r2_linearReg = r2_score(y_test, y_pred)

# Print results
print(f"Mean Absolute Percentage Error (MAPE) = {mape}")
print(f"Mean Absolute Error (MAE) = {mae}")
print(f"Mean Squared Error (MSE) = {mse}")
print(f"Root Mean Squared Error (RMSE) = {rmse}")
print(f"R2 = {r2_linearReg}")
#Accuracy of the model
print(f"Accuracy of the model = {r2_score(y_test, y_pred)*100}")
```

Figure 4.4.2.0 Code For Linear Regression

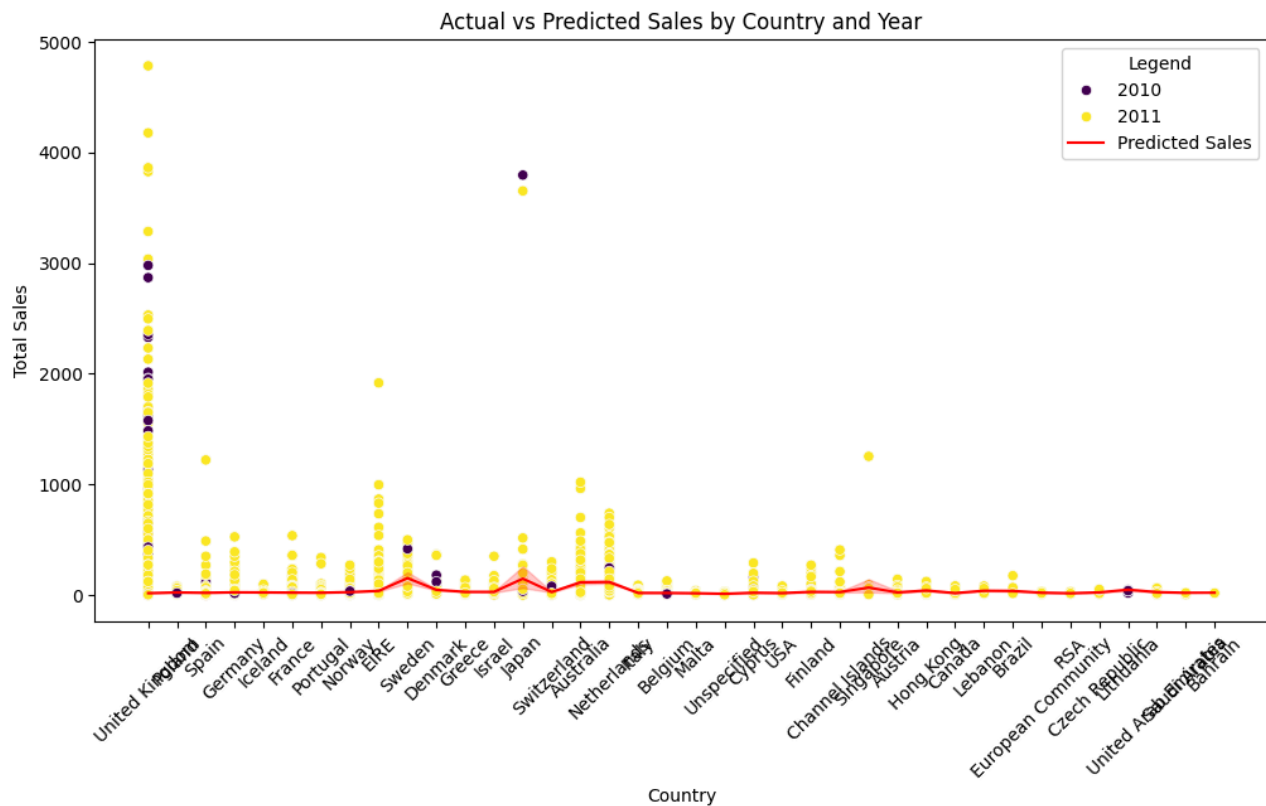


Figure 4.4.2.1 Plot Diagram

For the features which are variable, we are choosing the Quantity, Unit Price, Month, Year, Hour and Country. The target variable is to predict the Total Sales. Based on the **Figure 4.4.2.0**, we are using the ColumnTransformer to process data. For all the numeric features (Quantity, Unit Price, Month, Year and Hour) are standardized using StandardScaler, while for the categorical features (Country) are using OneHotEncoder to encode it. Next, we are using a pipeline that includes data preprocessors and models. Preprocessor is to apply data preprocessing steps such as processing all the data features, while model is to apply the LinearRegression model. After that, we are using the train-test_split to separate the data into test set and train set. We are set that the train set is 80% while the test set is 20%. At the end, we fit the pipeline using the training data.

4.4.3 Model Evaluation

```
Mean Absolute Percentage Error (MAPE) = 223547774766295.47
Mean Absolute Error (MAE) = 10.856241212809302
Mean Squared Error (MSE) = 2319.0568515805867
Root Mean Squared Error (RMSE) = 48.15658679329949
R2 = 0.4624276013738249
Accuracy of the model = 46.24276013738249
```

Figure 4.4.3.0

Mean Square Error

Mean Square Error (MSE) measures the accuracy of the model by calculating the square of the difference between the predicted and actual values and taking the average of these squared differences. The smaller the MSE, the closer the model's prediction to the actual value. Based on *Figure 4.4.3.0*, the Mean Squared Error (MSE) that we get is 2319.0568515805867. This represents the mean of the forecast error in squared terms.

Mean Absolute Percentage Error

Mean Absolute Percentage Error (MAPE) is measures of forecast accuracy that are often used to evaluate the accuracy of forecasting methods. It is to help to understand the deviation between predictions and actual conditions. Based on *Figure 4.4.3.0*, the MAPE value is 223547774766295.47. This value is very large, normally the MAPE value is under 10% shows high accuracy, 10-20% shows good forecast results, 20-50% shows reasonable forecast results, and greater than 50% shows incorrect results. So these results indicate that there may be data anomalies or calculation errors.

R-squared

R-squared (R^2) measures the ability of the model to explain the variability of the target variable. Based on *Figure 4.4.3.0*, the R^2 values is 0.4624276013738249. Therefore, this indicates that the model has weak explanatory power on the target variable and may not be the best choice for predicting our aim.

Root Mean Square Error

Root Mean Square Error (RMSE) is the square root of MSE, it provides the standard deviation of the forecast error. Based on *Figure 4.4.3.0*, the RMSE value is 48.15658679329949 means that the model's prediction error is approximately 48.15658679329949 which indicates that the model prediction accuracy is relatively low.

Mean Absolute Error

Mean Absolute Error (MAE) represents the average absolute difference between predicted and actual values.

The MAE value is low, indicating that the average error of the model in prediction is 10.856241212809302.

It indicates the error of the prediction result near the actual values.

4.4.4 Hyperparameter Tuning

```
from sklearn.model_selection import RandomizedSearchCV
from sklearn.linear_model import Ridge
from sklearn.pipeline import Pipeline
from sklearn.metrics import r2_score
from scipy.stats import uniform

# Create a pipeline for Ridge with preprocessing
pipeline_ridge = Pipeline(steps=[
    ('preprocessor', preprocessor),
    ('model', Ridge())
])

# Define the parameter grid for Ridge
param_ridge = {'model__alpha': [0.1, 1.0, 10.0, 100.0]}

# Perform RandomizedSearchCV for Ridge
random_search_ridge = RandomizedSearchCV(pipeline_ridge, param_distributions=param_ridge, n_iter=4, cv=5, random_state=42)
random_search_ridge.fit(X_train, y_train)

# Get the best model for Ridge
best_ridge = random_search_ridge.best_estimator_

# Make predictions with the best Ridge model
y_pred_ridge = best_ridge.predict(X_test)

# Calculate the R2 for Ridge
r2_ridge = r2_score(y_test, y_pred_ridge)
print(f"R2 score for Ridge: {r2_ridge:.4f}")
```

Figure 4.4.4.0

Based on **Figure 4.4.4.0**, we are using the Lasso regression to improve the model performance. First, we are using the pipeline to combine the data preprocessing and Lasso regression model. To define the hyperparameters of the Lasso Regression model, we adjust the alpha parameter[0.1,1.0,10.0,100.0], which controls the strength of the regularization. Next we use the RandomizedSearchCV to randomly search the hyperparameters. We set the cross-validation(cv=5) to evaluate the performance of each set of parameters. Choosing the best model from the random search and using the best model to predict the X_test. Last, we calculated the coefficient of determination which is R² of the best Lasso model to be used to measure the explanatory power of the model.

4.4.5 Mode Re-evaluation

R² score for Ridge: 0.4690

Figure 4.4.5.0

So by using the Lasso model, we get the R² is 0.4624 while before the hyperparameter tuning is 0.4690. Which only increased a few. Even after adjusting the hyperparameters tuning, the R² value changes remain small. It is indicating that the model is still limited in its ability to explain the variability of the target variable. The R² value of 0.46 indicates that the model explains only 46% of the variability in the target variable. Therefore, even if the hyperparameters are optimized, the performance improvement of the linear regression model is still insufficient and it may not be the best choice for predicting our target variable.

4.4 Decision Tree Algorithm

4.4.1 Algorithms and Assumptions

Algorithms

Regression decision tree algorithm is a kind of decision tree that predicts continuous numerical values instead of categorical ones. This is an outline of how it functions: The feature (or any other criterion linked to the target variable) that yields the largest decrease in variance is used by the algorithm to divide the data into subsets. Recursively repeating this technique creates a tree structure.

Assumptions of Decision Tree Algorithms

1. Decision trees are generally robust to outliers because they partition the data into subsets and make decisions based on these partitions.
2. Decision trees assume that the training data is representative of the data you expect in practice.

4.4.2 Model Building

```
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeRegressor
from sklearn.metrics import accuracy_score
from sklearn import tree

# Features and target variable
X = dfNew[['Quantity', 'UnitPrice', 'month', 'year', 'Country', 'Description']] # Features
y = dfNew['Total_Sales'] # Target variable

# Define the preprocessor with both scaling and encoding
preprocessor = ColumnTransformer(
    transformers=[
        ('num', StandardScaler(), ['Quantity', 'UnitPrice', 'month', 'year']),
        ('cat', OneHotEncoder(handle_unknown='ignore'), ['Country', 'Description'])
    ])

# Create the pipeline with preprocessing and model
pipeline_decisionTree = Pipeline(steps=[
    ('preprocessor', preprocessor),
    ('model', DecisionTreeRegressor(max_depth=15, min_samples_split=10, min_samples_leaf=6, random_state=42))
])

# Split the data into training and testing data set
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Create and train the Decision Tree model using the training data
pipeline_decisionTree.fit(X_train, y_train)

# Evaluate the model's performance using the testing data

## Make predictions on the test data
y_pred = pipeline_decisionTree.predict(X_test)

# Evaluate the model
mape = mean_absolute_percentage_error(y_test, y_pred)
mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
rmse = sqrt(mse)
r2_decision_tree = r2_score(y_test, y_pred)

# Print result
print(f"Mean Absolute Percentage Error (MAPE) = {mape}")
print(f"Mean Absolute Error (MAE) = {mae}")
print(f"Mean Square Error (MSE) = {mse}")
print(f"Root Mean Square Error (RMSE) = {rmse}")
print(f"R2 = {r2_decision_tree}")
```

Figure 4.4.2.0 Code For Decision Tree

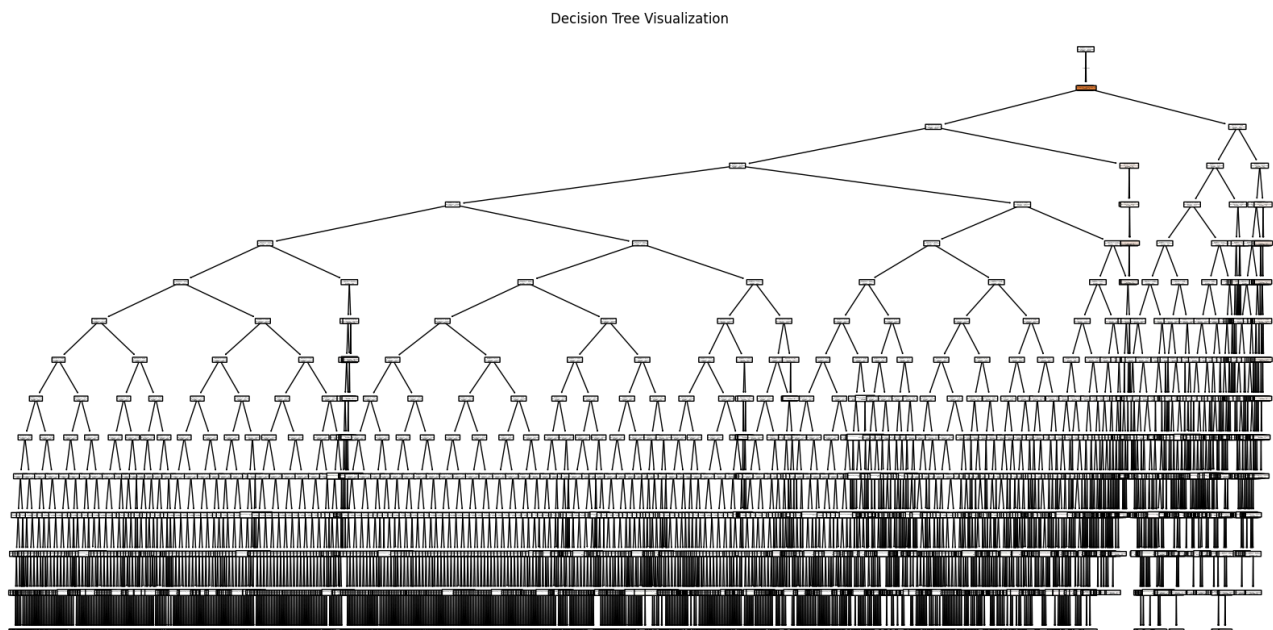


Figure 4.4.2.1 Plot Diagram

Based on the code given above, The features and target variables are the same. We are also using the same method but to differentiate it from the linear regression code we need to set the `max_depth`, `min_samples_split` and `min_samples_leaf`. The `max_depth` is a parameter to control the maximum depth of the tree, `min_samples_split` is to control the minimum number of the samples that are required to split and

`min_samples_leaf` is the minimum number of samples required to be at a leaf node. In our dataset, we set that the `max_depth = 15`, `min_samples_split = 10` and `min_samples_leaf = 6`. We have to make this setting so that our device will not take a very long time to produce the outcome for the plot diagram and the details of the model evaluation.

4.4.3 Model Evaluation

```
Mean Absolute Percentage Error (MAPE) = 8857244240991.477
Mean Absolute Error (MAE) = 0.5349245277539272
Mean Square Error (MSE) = 595.6804704999154
Root Mean Square Error (RMSE) = 24.406566134954655
R2 = 0.8619174087417665
```

Figure 4.4.3.0

Mean Square Error

Based on the *Figure 4.4.3.0*, the Mean Square Error (MSE) value that we get from the code is 595.6803984836338

Mean Absolute Percentage Error

Based on the *Figure 4.4.3.0*, the Mean Absolute Percentage Error (MAPE) value that we get from the code is 8857244240991.477

R-squared

Based on the *Figure 4.4.3.0*, the R-squared (R^2) value that we get from the code is 0.861917425435607

Root Mean Square Error

Based on the *Figure 4.4.3.0*, the Root Mean Square Error (RMSE) value that we get from the code is 24.406564659608154

Mean Absolute Error

Based on the *Figure 4.4.3.0*, the Mean Absolute Error (MAE) value that we get from the code is 0.5348661812295977

4.5 Random Forest Regression

4.5.1 Algorithms and Assumptions

Algorithms

A Random Forest Regression model is a single model that is made up of several decision trees. Based on distinct subsets of data, every tree in the forest generates its own autonomous forecast. Based on the average, or weighted average, of all the predictions made by each individual tree, the final input prediction is determined.

Assumptions of Linear Random Forest Regression Algorithms

1. While individual trees in the Random Forest are assumed to be less correlated due to the random sampling of data and features, it is not strictly assumed that the trees are completely independent.
2. The algorithm assumes that not all features need to be relevant for making accurate predictions.
3. Random Forest assumes that combining multiple trees through bagging (bootstrap aggregation) and averaging their predictions will reduce overfitting compared to a single decision tree.

4.5.2 Model Building

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_squared_error, mean_absolute_error, mean_absolute_percentage_error, r2_score
from math import sqrt

# Assuming dfNew is your DataFrame
# Features and target variable
X = dfNew[['Quantity', 'UnitPrice', 'month', 'year', 'Country', 'Description']] # Features
y = dfNew['Total_Sales'] # Target variable

# Split the data into training and testing data set
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Define the preprocessor with both scaling and encoding
preprocessor = ColumnTransformer(
    transformers=[
        ('num', StandardScaler(), ['Quantity', 'UnitPrice', 'month', 'year']),
        ('cat', OneHotEncoder(handle_unknown='ignore'), ['Country', 'Description'])
    ]
)

# Create the pipeline with preprocessing and RandomForestRegressor
pipeline_rf = Pipeline(steps=[
    ('preprocessor', preprocessor),
    ('model', RandomForestRegressor(max_depth=15, min_samples_split=10, min_samples_leaf=6, random_state=42))
])
```

```
# Train the model using the training data
pipeline_rf.fit(X_train, y_train)

# Generate predictions using the test data
y_pred = pipeline_rf.predict(X_test)

# Evaluate the model
mape = mean_absolute_percentage_error(y_test, y_pred)
mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
rmse = sqrt(mse)
r2_random_forest = r2_score(y_test, y_pred)

# Print results
print(f"Mean Absolute Percentage Error (MAPE) = {mape}")
print(f"Mean Absolute Error (MAE) = {mae}")
print(f"Mean Square Error (MSE) = {mse}")
print(f"Root Mean Square Error (RMSE) = {rmse}")
print(f"R2 = {r2_random_forest}")
```

Figure 4.5.2.0 Code For Random Forest

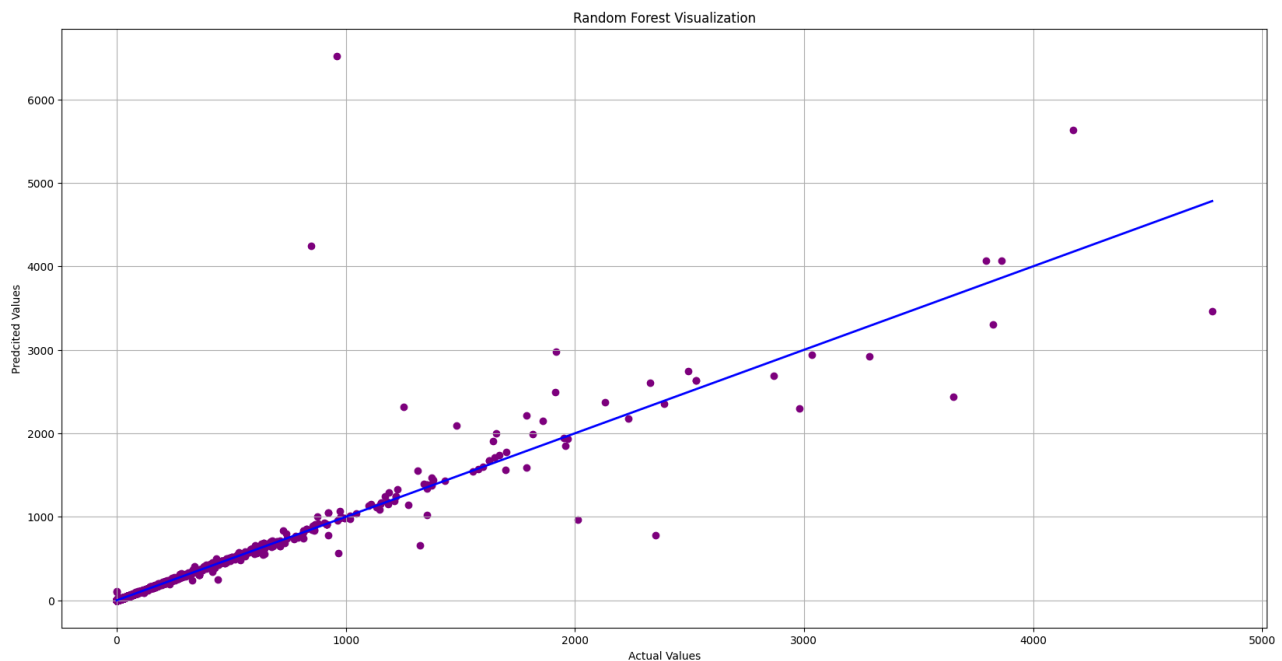


Figure 4.5.2.1 Plot Diagram

Based on the code given above, The features and target variables are the same. We are also using the same method but to differentiate it from the linear regression code we need to set the `max_depth`,

min_samples_split and min_samples_leaf. The max_depth is a parameter to control the maximum depth of the tree, min_samples_split is to control the minimum number of the samples that are required to split and min_samples_leaf is the minimum number of samples required to be at a leaf node. In our dataset, we set that the max_depth = 15, min_samples_split = 10 and min_samples_leaf = 6. We have to make this setting so that our device will not take a very long time to produce the outcome for the plot diagram and the details of the model evaluation.

4.5.3 Model Evaluation

```
Mean Absolute Percentage Error (MAPE) = 11123023442082.354
Mean Absolute Error (MAE) = 0.40298291529594693
Mean Square Error (MSE) = 541.9621455164978
Root Mean Square Error (RMSE) = 23.280080444802973
R2 = 0.8743696644041641
```

Figure 4.5.3.0

Mean Square Error

Based on the *Figure 4.5.3.0*, the Mean Square Error (MSE) value that we get from the code is 595.6803984836338

Mean Absolute Percentage Error

Based on the *Figure 4.5.3.0*, the Mean Absolute Percentage Error (MAPE) value that we get from the code is 8857244240991.477

R-squared

Based on the *Figure 4.5.3.0*, the R-squared (R^2) value that we get from the code is 0.861917425435607

Root Mean Square Error

Based on the *Figure 4.5.3.0*, the Root Mean Square Error (RMSE) value that we get from the code is 24.406564659608154

Mean Absolute Error

Based on the *Figure 4.5.3.0*, the Mean Absolute Error (MAE) value that we get from the code is 0.5348661812295977

5.0 Evaluation

5.1 Comparisons

	Linear Regression Algorithm	Decision Tree Algorithm	Random Forest Algorithm
Mean Square Error	2319.0490989586365	595.6803984836338	541.9621455164978
Mean Absolute Percentage Error	223595044119942.94	8857244240991.477	11123023442082.354
R-squared	0.4624293984818063	0.861917425435607	0.8743696644041641
Root Mean Square Error	48.15650629934274	24.406564659608154	23.280080444802973
Mean Absolute Error	10.856117341637182	0.5348661812295977	0.40298291529594693

Table 5.1.0

5.2 Best Model Performance

Based on *Table 5.1.0*, by comparing all the model evaluations, we can find that the Random Forest Algorithm is the best performance. This is because that all the evaluation such as the Mean Square Error (MSE) is the lowest, Mean Absolute Percentage Error(MAPE) is the lowest, R^2 is the highest, Root Mean Square Error is the lowest and the Mean Absolute Error(MAPE) is the lowest.

The R^2 of the Random Forest Algorithm is 0.8743696644041641, which means that the accuracy score is 87.44% after rounded up, it is the highest R^2 among the 3 algorithms shown above.

6.0 Deployment

Input Parameters

Select Countries

United Kingdom

Switzerland

Description

WHITE HANGING HEART T-LIGH...

Quantity

10

Unit Price

2.55

Years

2022

2024

Months

1

7

9

10

11

12

Predict Sales

Sales Prediction App

Prediction Comparison by Country

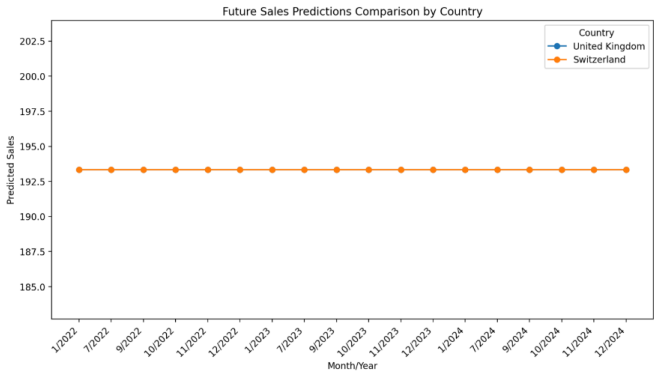


Figure 6.0.0

The term “model bias” describes the systematic mistakes in a machine learning model that result from underlying presumptions or incomplete data. Based on **Figure 6.0.0**, we can see that the model prefers to learn data from “United States” compared to “Switzerland” because it has more samples than “Switzerland”. Due to the large proportion of “United Kingdom” data, this may cause the model to predict it more accurately and predict “Switzerland” less accurately. In the actual prediction, if the input is “Switzerland”, the model may not have enough data to accurately predict the sales in that country, which may lead to a biased prediction.

7.0 Conclusions

Based on 4.0 and 5.0 of the entire assignment, we conclude that 3 of our models, which are the linear regression model, decision tree model and random forest model. These 3 models have their respective pros and cons that we have to consider when facing specific circumstances. We should consider carefully about their ups and downs in order to determine the best usage in different scenarios even if one model's evaluation is better than the other.

Linear Regression Model

A linear regression model predicts the target variable based on the linear relationship between the target variable and input variables. One of the first advantages of linear regression models is that the model is easy to understand and interpret with clear coefficients that represent the contribution of each variable to the target variable. Moreover, the linear regression model is able to work well with a dataset consisting of fewer features due to its computation efficiency. Just like the name suggested, It also performs very well especially when the relationship between variables is truly linear. Due to its predictive power, it can make quick predictions with decent accuracy for simple relationships. Even with all these advantages, it still has some limitations that we should consider, such as the very fact that it can only be used on target and features that have a linear relationship between them. Another limitation to mentioned is that linear regression is very sensitive to outliers, which can skew the predictions

Decision Tree Model

A decision tree is a flowchart-like structure where nodes represent decisions based on features, leading to predictions at the leaf nodes. There are numerous advantages for using the decision tree model, one of being its high interpretability due to the fact that the decision path can be visualised. This would ease the process of explaining to non-technical stakeholders. Unlike linear regression models, it can capture non-linear relationships between variables effectively. It also does not require normalisation or standardisation of data. The decision tree model is able to work well with both numerical and categorical data. With all these advantages, there are still some weaknesses that we should also be aware of. Weaknesses such as decision trees are prone to overfitting especially in cases where the tree becomes too deep. Some small changes in the data will produce a completely different tree structure. There is also bias in cases where decision trees can have high variance and might prefer features with more levels or continuous variables. Its predictive power is quite low since a single decision tree often provides low predictive accuracy compared to other models

Random Forest Model

Random forest is an ensemble learning technique that builds multiple decision trees and aggregates their results for prediction. The fact that it aggregates results of multiple decision trees, helps in reducing the risk of overfitting since multiple trees are averaged while at the same time increasing accuracy. Just like decision trees, random forests are able to capture non-linear relationships and feature interactions. Random forests are able to handle missing data effectively. On the other hand, due to the fact that random forests aggregate predictions of multiple trees, they are less interpretable than other models. It can also be time consuming if the dataset is quite large as it takes a lot of time to build and make predictions. It also consumes a lot of memory when the dataset is quite large. During the tree-splitting process, random forests tend to favour features with more levels or continuous features.

In summary, the best model for a given issue depends on the specific characteristics of your data and the goal of your analysis you set to achieve. It is always recommended to experiment with different models and evaluate each of their performance given the circumstances and to find the best model suited for the job.

8.0 References

1. [Assignment\[Final\].ipynb](#)
2. <https://www.geeksforgeeks.org/hyperparameter-tuning-in-linear-regression/> (Jain, 2024)
3. <https://www.google.com/url?q=https%3A%2F%2Fmedium.com%2F%40diehardankush%2Fhow-implementing-decision-trees-in-python-with-scikit-learn-part-3-29e5a787baaf> (Singh, 2023)
4. <https://medium.com/@tubelwj/how-to-implement-the-random-forest-algorithm-by-python-51995cabf8a5> (*How to Implement the Random Forest Algorithm by Python*, 2023)

10.0 Appendix

10.1.0 Coding Part for Data Understanding

10.1.1 Data Loading

```
[55] import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
df = pd.read_csv('OnlineRetail.csv')
df
```

10.1.2 Descriptive Statistics

```
# Get statistics of the quantity in dataframe
print("Unit Price and Quantity statistics")
print("=====")
describe_columns = ['UnitPrice', 'Quantity']
subset = df[describe_columns]

# Get summary statistics for the selected columns
stat = subset.describe()
stat
```

```
# get the mode and median of the quantity and unit price
print("Mode of Unit Price & Quantity")
print("=====")
mode = subset.mode()
mode
```

```
print("Median of Unit Price and Quantity")
print("=====")
median = subset.median()
median
```

10.1.3 Data Exploration

10.1.3.1 Relationship Exploration and Visualization using Heatmap

```
# Heatmap of all variable
# need to run the first code again if got error
df['Sales'] = df['UnitPrice'] * df['Quantity']
df['InvoiceDate'] = pd.to_datetime(df['InvoiceDate'])

df['Year'] = df['InvoiceDate'].dt.year
df['Month'] = df['InvoiceDate'].dt.month

# find the relationship
corr_matrix = df[['Quantity', 'UnitPrice', 'Sales', 'Year', 'Month', 'Hour']].corr()
plt.figure(figsize=(10, 8))
sns.heatmap(corr_matrix, annot=True, cmap='coolwarm', linewidths=1.0, linecolor='white')
plt.title('Heatmap Distribution')
plt.show()
```

10.1.3.2 Scatter Plot of Unit Price over Sales

```
# Scatter plot for unit price and sales
plt.figure(figsize=(10, 6)) # Optional: Set the figure size

# Plotting the scatter plot
plt.scatter(df['UnitPrice'], df['Sales'], color='lime', alpha=0.7, edgecolors='w', s=100)

# Adding titles and labels
plt.title('Scatter Plot of Unit Price vs Sales')
plt.xlabel('Unit Price')
plt.ylabel('Sales')

# Adding grid
plt.grid(True)

# Show the plot
plt.show()
```

10.1.3.3 Distribution Exploration and Visualisation using Pie Chart

```
labels = ['Cancelled Transactions', 'Not Cancelled Transactions']
sizes = [len(cancelled_transaction), len(df)-len(cancelled_transaction)]

# Function to format the labels
def func(pct, allvals):
    absolute = int(round(pct / 100. * sum(allvals)))
    return f'{absolute} ({pct:.1f}%)'

# Create the pie chart
plt.figure(figsize=(8, 6))
plt.pie(sizes, labels=labels, autopct=lambda pct: func(pct, sizes), startangle=140, colors=['#ff9999', '#66b3ff'])

# Add a title
plt.title('Transaction Status Distribution')

# Ensure the pie chart is a circle
plt.axis('equal')

# Show the plot
plt.show()
```

10.1.3.4 Bar Chart of Transaction made by Day of Week

```
invoice_weekday = pd.DataFrame(df.groupby(['WeekDay'])['InvoiceNo'].count()).reset_index()
# Create the bar plot using Matplotlib
plt.figure(figsize=(10, 6))
bars = plt.bar(invoice_weekday['WeekDay'], invoice_weekday['InvoiceNo'], color='violet')

# Customize the plot
plt.title('Transaction by Day of Week')
plt.xlabel('Day of Week')
plt.ylabel('Invoice Number')
plt.xticks(rotation=45) # Rotate x-axis labels for better readability
plt.grid(axis='y') # Add grid lines on the y-axis

# Add values on top of each bar
for bar in bars:
    height = bar.get_height()
    plt.text(bar.get_x() + bar.get_width() / 2.0, height, f'{height}',
             ha='center', va='bottom', fontsize=10, color='black')

# Show the plot
plt.tight_layout()
plt.show()
```

10.1.3.5 Bar Chart of Number of Transaction Made According to Hour

```
invoice_no = pd.DataFrame(df.groupby(['Hour'])['InvoiceNo'].count()).reset_index()
# Create a bar chart using Matplotlib
plt.figure(figsize=(10, 6))
bars = plt.bar(invoice_no['Hour'], invoice_no['InvoiceNo'], color='skyblue')

# Customize the plot
plt.title('Transaction by Hour')
plt.xlabel('Hour of the Day')
plt.ylabel('Transaction Made')
plt.xticks(rotation=45) # Rotate x-axis labels for better readability
plt.grid(axis='y') # Add grid lines along the y-axis

# Add values on top of each bar
for bar in bars:
    height = bar.get_height()
    plt.text(bar.get_x() + bar.get_width() / 2.0, height, f'{height}',
             ha='center', va='bottom', fontsize=10, color='black')

# Show the plot
plt.tight_layout()
plt.show()
```

10.1.3.6 Line Graph of Number of Invoices over Time

```
plot = pd.DataFrame(df.groupby(['month_year'])['InvoiceNo'].count()).reset_index()
# Create the line plot using Matplotlib
plt.figure(figsize=(10, 6))
plt.plot(plot['month_year'], plot['InvoiceNo'], marker='o', linestyle='-', color='blue')

# Customize the plot
plt.title('Number of Invoice Over Time ')
plt.xlabel('Month and Year')
plt.ylabel('Number of Invoice')
plt.grid(True)
plt.xticks(rotation=45) # Rotate x-axis labels for better readability

# Add values on each data point
for i, row in plot.iterrows():
    plt.text(row['month_year'], row['InvoiceNo'], str(row['InvoiceNo']),
            ha='center', va='bottom', fontsize=10, color='grey')

# Show the plot
plt.tight_layout()
plt.show()
```

10.1.3.7 Line Chart of Sales over Time

```
sales_time= pd.DataFrame(df.groupby(['month_year'])['Sales'].sum()).reset_index()
# Create a line plot using Matplotlib
plt.figure(figsize=(10, 6))
plt.plot(sales_time['month_year'], sales_time['Sales'], marker='o', linestyle='-', color='green')

for i in range(len(sales_time)):
    plt.text(sales_time['month_year'][i], sales_time['Sales'][i],
            f'{sales_time["Sales"][i]}',
            ha='center', va='bottom', fontsize=6, color='grey')

# Customize the plot
plt.title('Sales over Time')
plt.xlabel('Month-Year')
plt.ylabel('Sales')
plt.xticks(rotation=45) # Rotate x-axis labels for better readability
plt.grid(True) # Add grid lines for better readability

# Show the plot
plt.tight_layout()
plt.show()
```

10.1.3.8 Bar Chart of Top 10 Items Sold by Quantity

```
## Aggregate the total quantity sold by item description
item_quantity = pd.DataFrame(df.groupby('Description')['Quantity'].sum()).reset_index()

# Sort the DataFrame by 'Quantity' in descending order
sorted_item_quantity = item_quantity.sort_values(by='Quantity', ascending=False)

# Select the top 10 items by quantity sold
top10_items = sorted_item_quantity.head(10)

# Create the bar plot
plt.figure(figsize=(12, 7))
bars = plt.bar(top10_items['Description'], top10_items['Quantity'], color='pink')

# Annotate each bar with its value
for bar in bars:
    height = bar.get_height()
    plt.text(
        bar.get_x() + bar.get_width() / 2, # X position
        height, # Y position (height of the bar)
        f'{int(height)}', # Text value
        ha='center', # Horizontal alignment
        va='bottom', # Vertical alignment
        fontsize=10, # Font size
        color='black' # Text color
    )

# Customize the plot
plt.title('Top 10 Items by Quantity Sold')
plt.xlabel('Item Description')
plt.ylabel('Quantity Sold')
plt.xticks(rotation=45, ha='right') # Rotate x-axis labels for better readability
plt.grid(axis='y', linestyle='--', alpha=0.7) # Add grid lines along the y-axis

# Show the plot
plt.tight_layout()
plt.show()
```

10.1.3.9 Bar Chart of Top 10 Items Sold by Sales

```
# Aggregate total sales by item description
item_sales = df.groupby('Description')['Sales'].sum().reset_index()

# Find the top 10 items by sales
top10_items = item_sales.nlargest(10, 'Sales')

# Create the bar chart
plt.figure(figsize=(12, 8))
bars = plt.bar(top10_items['Description'], top10_items['Sales'], color='turquoise')
# Annotate each bar with its value
for bar in bars:
    height = bar.get_height()
    plt.text(
        bar.get_x() + bar.get_width() / 2, # X position (center of the bar)
        height, # Y position (on top of the bar)
        f'{int(height)}', # Text value
        ha='center', # Horizontal alignment
        va='bottom', # Vertical alignment
        fontsize=10, # Font size
        color='black' # Text color
    )

# Customize the plot
plt.title('Top 10 Items by Sales')
plt.xlabel('Item Description')
plt.ylabel('Sales')
plt.xticks(rotation=45, ha='right') # Rotate x-axis labels for better readability
plt.grid(axis='y') # Add grid lines along the y-axis

# Show the plot
plt.tight_layout()
plt.show()
```

10.1.3.10 Bar Chart of Top 10 Sales by Customer

```
# Group by 'CustomerID' and sum the sales
sales_by_customer = df.groupby('CustomerID')['Sales'].sum()

# Sort the results in descending order
top_sales_cust = sales_by_customer.sort_values(ascending=False)

# Select the top 10 entries
top_10_sales3 = top_sales_cust.head(10)

plt.figure(figsize=(10, 6)) # Optional: Set the figure size

# Plotting the bar chart
bar = plt.bar(top_10_sales3.index.astype(str), top_10_sales3.values, color='brown')

# Adding titles and labels
plt.title('Top 10 Sales by CustomerID')
plt.xlabel('CustomerID')
plt.ylabel('Total Sales')

# Rotate x-axis labels if needed for readability
plt.xticks(rotation=45)

# Add grid lines for better readability
plt.grid(axis='y', linestyle='--', alpha=0.7)

# Adjust layout to fit labels
plt.tight_layout()

# Show the plot
plt.show()
```

10.1.3.11 Bar Chart of Top 5 Sales by Country

```
## Aggregate the total quantity sold by item description
item1_Sales = df.groupby('Country')['Sales'].sum().reset_index()

# Find the top 5 items by quantity sold
top5_items = item1_Sales.nlargest(5, 'Sales')

# Create the bar plot
plt.figure(figsize=(12, 7))
bars = plt.bar(top5_items['Country'], top5_items['Sales'], color='darkblue')

# Annotate each bar with its value
for bar in bars:
    height = bar.get_height()
    plt.text(
        bar.get_x() + bar.get_width() / 2, # X position
        height, # Y position (height of the bar)
        f'{int(height)}', # Text value
        ha='center', # Horizontal alignment
        va='bottom', # Vertical alignment
        fontsize=10, # Font size
        color='black' # Text color
    )

# Customize the plot
plt.title('Top 5 Sales by Country')
plt.xlabel('Country')
plt.ylabel('Sales')
plt.xticks(rotation=45, ha='right') # Rotate x-axis labels for better readability
plt.grid(axis='y', linestyle='--', alpha=0.7) # Add grid lines along the y-axis

# Show the plot
plt.tight_layout()
plt.show()
```

10.1.3.12 Bar Chart of Bottom 5 Sales by Country

```
[ ] ## Aggregate the total quantity sold by country
item2_Sales = df.groupby('Country')['Sales'].sum().reset_index()

# Find the top 5 items by quantity sold
bottom5_items = item2_Sales.nsmallest(5, 'Sales')

# Create the bar plot
plt.figure(figsize=(12, 7))
bars = plt.bar(bottom5_items['Country'], bottom5_items['Sales'], color='lightblue')

# Annotate each bar with its value
for bar in bars:
    height = bar.get_height()
    plt.text(
        bar.get_x() + bar.get_width() / 2, # X position
        height, # Y position (height of the bar)
        f'{int(height)}', # Text value
        ha='center', # Horizontal alignment
        va='bottom', # Vertical alignment
        fontsize=10, # Font size
        color='black' # Text color
    )

# Customize the plot
plt.title('Bottom 5 Sales by Country')
plt.xlabel('Country')
plt.ylabel('Sales')
plt.xticks(rotation=45, ha='right') # Rotate x-axis labels for better readability
plt.grid(axis='y', linestyle='--', alpha=0.7) # Add grid lines along the y-axis

# Show the plot
plt.tight_layout()
plt.show()
```

10.1.4 Data Quality Verification

10.1.4.1 Detect of Outliers

```
# Check outliers
# Get statistics of the quantity in dataframe
print("Measurement of central tendency and statistics")
print("=====")
find_outliers = ['UnitPrice', 'Quantity', 'CustomerID', 'Sales']
col1 = df[find_outliers]

# Get summary statistics for the selected columns
outliers = col1.describe()
outliers
```

10.1.4.2 Verification of Data Types

```
# verify data types
data_types = df.dtypes
data_types
```

10.1.4.3 Detect of Missing Values

```
# Check Missing Values
null_values = df.isnull()
null_values

# Check whether have missing values in all columns in the dataset
null = df.isnull().any()
null

# total numbers of null values in the dataset
ttl_null = df.isnull().sum()
ttl_null
```

10.1.4.4 Detect of Duplicate Rows

```
# Check Duplicate Rows
duplicates_rows = df[df.duplicated()]

print("Numbers of duplicates rows :" + str(duplicates_rows.shape[0]))
duplicates_rows
```

10.1.4.5 Verify and Validate Numeric Ranges

```
# validate numeric ranges - quantity
invalid_quantity = df[(df['Quantity'] <= 0) | (df['Quantity'] > 10000)]
print("Number of invalid numeric ranges according to quantity : " + str(invalid_quantity.shape[0]))
invalid_quantity
```

```
# validate numeric ranges - unitprice
invalid_unitprice = df[(df['UnitPrice'] <= 0) ]
print("Number of invalid numeric ranges according to Unit Price: " + str(invalid_unitprice.shape[0]))
invalid_unitprice
```

```
# validate numeric ranges - sales
invalid_sales= df[(df['Sales'] <= 0) ]
print("Number of invalid numeric ranges according to Sales: " + str(invalid_sales.shape[0]))
invalid_sales
```

10.1.5 Data Preparation

10.1.5.1 Data Selection

```
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.preprocessing import LabelEncoder

# Select the columns you want
select_columns = ['UnitPrice', 'Quantity', 'Total_Sales', 'Hour', 'year', 'month', 'Country']
df_selected = df[select_columns]

# Encode the 'Country' column to numeric values
label_encoder = LabelEncoder()
df_selected['Country'] = label_encoder.fit_transform(df_selected['Country'])

# Compute the Pearson correlation matrix
corr = df_selected.corr(method='pearson')

plt.figure(figsize=(8, 6))
sns.heatmap(corr, annot=True, cmap='coolwarm', vmin=-1, vmax=1, center=0, linewidths=0.5)
plt.title('Pearson Correlation Heatmap')
plt.show()
```

10.1.5.2 Data Cleaning

10.1.5.2.1 Clean quantity negative number

```
#Clean the Quantity that is have negative number
df_negativeQtyValue = print(df[df['Quantity']<0])
df_cleaned = df[df['Quantity'] >= 0]
print(df_cleaned)
```

10.1.5.2.2 Clean Unit Price

```
#Clean Unit Price
df_POST_DOT = print(df_cleaned[df_cleaned['UnitPrice']<0])
df_clean_QTY_UP = df_cleaned[df_cleaned['UnitPrice'] >= 0]
print(df_clean_QTY_UP)
```

```
#Clean Unit Price
df4 = pd.read_csv('Updated_Clean_dataset.csv', low_memory=False)
df_clean_QTYIS0 = df4[df4['UnitPrice'] > 0]
print(df_clean_QTYIS0)
# Save the updated DataFrame to a new CSV file
df_clean_QTYIS0.to_csv('Updated_Clean_dataset.csv', index=False)
```

```
#Copy the cleaned dataset to a csv file
df_clean_QTY_UP.to_csv('Clean_QTY_UP_dataset.csv', index=False, encoding='utf-8', na_rep='NA')

#Check that got updated or not#
df2 = pd.read_csv('Clean_QTY_UP_dataset.csv', low_memory=False)
print(df2)
```

10.1.5.2.3 Clean Dot, Post and Description is empty

```
#Find the DOT and POST in Stock Code#
df2 = pd.read_csv('Clean_QTY_UP_dataset.csv', low_memory=False)

df_clean_QTY_UP_DOT_POST = df2[df2['StockCode'].str.contains('DOT|POST', case=False, na=False)]
print(df_clean_QTY_UP_DOT_POST)
```

```
#Don't know why above the dot and post didn't remove, so remove here again#
check_DOT_POST = df[~df['StockCode'].str.contains('DOT|POST', case=False, na=False)]
print(check_DOT_POST)
# Save the updated DataFrame to a new CSV file
check_DOT_POST.to_csv('Updated_Clean_dataset.csv', index=False)
```

```
#Find the DOT and POST in Stock Code#
df2 = pd.read_csv('Clean_QTY_UP_dataset.csv', low_memory=False)
print(df2)

df_clean_QTY_UP_DOT_POST = df2[~df2['StockCode'].str.contains('DOT|POST', case=False, na=False)]
print(df_clean_QTY_UP_DOT_POST)

# save into current csv file
df_clean_QTY_UP_DOT_POST.to_csv('Clean_QTY_UP_dataset.csv', index=False)
```

```
#Check the description is empty#
# Read the CSV file into a DataFrame
df2 = pd.read_csv('Clean_QTY_UP_dataset.csv', low_memory=False)

# Filter rows where Description is empty
df_clean_QTY_UP_DOT_POST_DESC = df2[df2['Description'].isna()]

# Print the filtered DataFrame
print(df_clean_QTY_UP_DOT_POST_DESC)

# Print the number of rows with empty or whitespace-only Description
Totnum_rows_empty_description = df_clean_QTY_UP_DOT_POST_DESC.shape[0]
print(f"Number of rows with empty or whitespace-only Description: {Totnum_rows_empty_description}")
```

```
#Remove the empty description
df_clean_QTY_UP_DOT_POST_DESC = df2[df2['Description'].notna()]
print(df_clean_QTY_UP_DOT_POST_DESC)

# Print the number of rows
Totnum_rows_empty_description = df_clean_QTY_UP_DOT_POST_DESC.shape[0]
print(f"Number of rows: {Totnum_rows_empty_description}")

#Save to clean_file
df_clean_QTY_UP_DOT_POST_DESC.to_csv('Clean_QTY_UP_dataset.csv', index=False)
```

10.1.5.2.4 Replace Customer ID

```
#Check the Customer Id that is empty#
# Read the CSV file into a DataFrame
df3 = pd.read_csv('Clean_QTY_UP_dataset.csv', low_memory=False)

# Filter out rows where empty CustomerId
check_CUSTID_empty = df3[df3['CustomerID'].isna()]
print(check_CUSTID_empty)
```

```
# filter out the invoice no 536544 and it empty customer id
invoice_no = '536544'
invoice_data = df3[(df3['InvoiceNo'] == invoice_no) & (df3['CustomerID'].isna())]

# print the data to check
print(invoice_data)
```

```
import pandas as pd

# Load the dataset
df4 = pd.read_csv('Clean_QTY_UP_dataset.csv', low_memory=False)

# Filter out rows where CustomerID is empty
check_CUSTID_empty = df4[df4['CustomerID'].isna()]

# Extract all the InvoiceNo values where CustomerID is empty
empty_customer_invoice_nos = check_CUSTID_empty['InvoiceNo'].unique()

# Convert the InvoiceNo values to a string format for writing to file
invoice_nos_str = '\n'.join(empty_customer_invoice_nos)

# Save to a text file
with open('empty_customer_invoice_nos.txt', 'w') as file:
    file.write(invoice_nos_str)

print("Invoice numbers with empty CustomerID have been saved to 'empty_customer_invoice_nos.txt'.")
```

```

#Find a more better solution to input the customer ID#
# Read the CSV file into a DataFrame
df4 = pd.read_csv('Clean_QTY_UP_dataset.csv', low_memory=False)

# Read the invoice numbers from the text file
df5 = pd.read_csv('empty_customer_invoice_nos.txt', header=None)
invoice_nos = df5[0].tolist()

# Ensure the InvoiceNo column is treated as string type
df4['InvoiceNo'] = df4['InvoiceNo'].astype(str)

# Filter out rows where InvoiceNo is in the list and CustomerID is empty
invoice_data = df4[df4['InvoiceNo'].isin(invoice_nos) & df4['CustomerID'].isna()]

# print the data to check
print(invoice_data)

def check_customer_id_unique(df, customer_id):
    """Check if a given CustomerID is unique in the DataFrame."""
    return customer_id not in df['CustomerID'].values

# Generate new CustomerIDs starting from 10000
start_id = 10000
i = 5
new_customer_ids = [str(start_id + i) for i in range(len(invoice_nos))]

# Assign CustomerIDs to the corresponding InvoiceNos
for invoice_no, new_customer_id in zip(invoice_nos, new_customer_ids):
    if check_customer_id_unique(df4, new_customer_id):
        df4.loc[(df4['InvoiceNo'] == invoice_no) & (df4['CustomerID'].isna()), 'CustomerID'] = new_customer_id
        print(f"CustomerID {new_customer_id} has been successfully assigned to InvoiceNo {invoice_no}.")
    else:
        print(f"CustomerID {new_customer_id} already exists. Please choose a different one.")

# Print the updated DataFrame
print("Updated Invoice Data:")
print(df4[(df4['InvoiceNo'] == invoice_no)])

# Save the updated DataFrame to a new CSV file
df4.to_csv('Updated_Clean_dataset.csv', index=False)

```

```

#Check the data that after input the CustomerID got release or not
df = pd.read_csv('Updated_Clean_dataset.csv', low_memory=False)

# Filter out rows where empty CustomerID
check_CUSTID_empty = df[df['CustomerID'].isna()]
print(check_CUSTID_empty)

# Extract all the InvoiceNo values where CustomerID is empty
empty_customer_invoice_nos = check_CUSTID_empty['InvoiceNo'].unique()

# Print the InvoiceNo values
print("Invoice numbers with empty CustomerID:")
print(empty_customer_invoice_nos)

```

```
#Add another column (variable) Total Sales#
if 'Quantity' in df.columns and 'UnitPrice' in df.columns:
    # Compute 'Total_Sales' as Quantity * Unit Price
    df['Total_Sales'] = df['Quantity'] * df['UnitPrice']
else:
    print("Required columns 'Quantity' and/or 'Unit Price' are missing from the dataset.")

# Save the updated DataFrame to a new CSV file
df.to_csv('Updated_Clean_dataset.csv', index=False)
print(df)
```

```
df['InvoiceDate'] = pd.to_datetime(df['InvoiceDate'], format="%Y-%m-%d %H:%M:%S")
df['month'] = df['InvoiceDate'].dt.month
df['year'] = df['InvoiceDate'].dt.year
df['WeekDay'] = df['InvoiceDate'].dt.day_name()
df['month_year'] = pd.to_datetime(df[['year', 'month']].assign(Day=1))
df['Hour'] = df['InvoiceDate'].dt.hour
# Save the updated DataFrame to a new CSV file
df.to_csv('Updated_Clean_dataset.csv', index=False)
print(df.head)
```

10.1.5.2.5 Remove Outlier

```
df = pd.read_csv('Updated_Clean_dataset.csv', low_memory=False)
df['InvoiceDate'] = pd.to_datetime(df['InvoiceDate'])
print(df.dtypes)
```

```
#Check the overall variable
plt.figure(figsize=(10,5))
df.boxplot(patch_artist=True, sym='k.')
plt.show()
```

```
from scipy import stats

#Print the z-score
z_score = np.abs(stats.zscore(df[['Quantity', 'UnitPrice', 'Total_Sales'])))
print(z_score)
```

```
#Print the z-score > 3#
z_score_limit = 3
outlier_sales = np.where(z_score > z_score_limit)
print(outlier_sales)
```

```
#Remove outliers#
df7=df[(z_score < z_score_limit).all(axis=1)]
# # Save the updated DataFrame to a new CSV file
# df7.to_csv('Updated_Clean_dataset.csv', index=False)

plt.figure(figsize=(10,5))
df7.boxplot(patch_artist=True, sym='k.')
plt.show()
```

10.1.6 Data Construction

```
# dfNew = pd.read_csv('Updated_Clean_dataset.csv', low_memory=False)
dfNew = df
print(dfNew)
```

```
dfNew['InvoiceDate'] = pd.to_datetime(dfNew['InvoiceDate'])
print(dfNew.dtypes)
```

10.1.6.1 Separate Categorical and continuous data

```
#Define that which is categorical and continuous
categorical_col = ['InvoiceNo', 'StockCode', 'Description', 'Country']
continuous_col = ['Quantity', 'UnitPrice', 'Total_Sales', 'month', 'year', 'Hour']

categorical_df = df[categorical_col]
continuous_df = df[continuous_col]
```

10.1.8 Data Normalisation

```
from sklearn.preprocessing import MinMaxScaler
scaler = MinMaxScaler()
normalized_continuous_df = pd.DataFrame(scaler.fit_transform(continuous_df), columns=continuous_col)

#Just for checking#

print(len(categorical_df))
print(len(normalized_continuous_df))

# Check indices
print(categorical_df.index)
print(normalized_continuous_df.index)

print(categorical_df.columns)
print(normalized_continuous_df.columns)

categorical_df = categorical_df.reset_index(drop=True)
normalized_continuous_df = normalized_continuous_df.reset_index(drop=True)
```

10.1.9 Data Concatenation

```
for col in categorical_col:
    categorical_df[col] = categorical_df[col].astype('category')

combine_df = pd.concat([categorical_df, normalized_continuous_df], axis=1)

#Result
print(combine_df.head())
```

10.1.10 Data Modeling

10.1.10.1 Linear Regression

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_absolute_percentage_error, mean_squared_error, r2_score
from math import sqrt

# Features and target variable
X = dfNew[['Quantity', 'UnitPrice', 'month', 'year', 'Country', 'Description']] # Features
y = dfNew['Total_Sales'] # Target variable

# Define the preprocessor with both scaling and encoding
preprocessor = ColumnTransformer(
    transformers=[
        ('num', StandardScaler(), ['Quantity', 'UnitPrice', 'month', 'year']),
        ('cat', OneHotEncoder(handle_unknown='ignore'), ['Country', 'Description'])
    ])

# Create the pipeline with preprocessing and model
pipeline = Pipeline(steps=[
    ('preprocessor', preprocessor),
    ('model', LinearRegression())
])

# Split the data into training and testing datasets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Fit the pipeline with the training data
pipeline.fit(X_train, y_train)

# Make predictions with the test data
y_pred = pipeline.predict(X_test)

# Evaluate the model
mape = mean_absolute_percentage_error(y_test, y_pred)
mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
rmse = sqrt(mse)
r2_linearReg = r2_score(y_test, y_pred)

# Print results
print(f"Mean Absolute Percentage Error (MAPE) = {mape}")
print(f"Mean Absolute Error (MAE) = {mae}")
print(f"Mean Squared Error (MSE) = {mse}")
print(f"Root Mean Squared Error (RMSE) = {rmse}")
print(f"R2 = {r2_linearReg}")
#Accuracy of the model
print(f"Accuracy of the model = {r2_score(y_test, y_pred)*100}")
```

10.1.10.1.1 Hyperparameter Tuning

```
from sklearn.model_selection import RandomizedSearchCV
from sklearn.linear_model import Ridge
from sklearn.pipeline import Pipeline
from sklearn.metrics import r2_score
from scipy.stats import uniform

# Create a pipeline for Ridge with preprocessing
pipeline_ridge = Pipeline(steps=[
    ('preprocessor', preprocessor),
    ('model', Ridge())
])

# Define the parameter grid for Ridge
param_ridge = {'model__alpha': [0.1, 1.0, 10.0, 100.0]}

# Perform RandomizedSearchCV for Ridge
random_search_ridge = RandomizedSearchCV(pipeline_ridge, param_distributions=param_ridge, n_iter=4, cv=5, random_state=42)
random_search_ridge.fit(X_train, y_train)

# Get the best model for Ridge
best_ridge = random_search_ridge.best_estimator_

# Make predictions with the best Ridge model
y_pred_ridge = best_ridge.predict(X_test)

# Calculate the R2 for Ridge
r2_ridge = r2_score(y_test, y_pred_ridge)
print(f"R2 score for Ridge: {r2_ridge:.4f}")
```

10.1.10.1.2 Model Revaluation

```
import matplotlib.pyplot as plt
import seaborn as sns

X_test_with_preds = X_test.copy() # Copy X_test to avoid altering original data
X_test_with_preds['Predicted_Sales'] = y_pred_lasso
X_test_with_preds['Actual_Sales'] = y_test.values

# Plotting
plt.figure(figsize=(12, 6))

# Scatter plot for Actual Sales
sns.scatterplot(x='Country', y='Actual_Sales', hue='year', data=X_test_with_preds, palette='viridis')

# Line plot for Predicted Sales
sns.lineplot(x='Country', y='Predicted_Sales', data=X_test_with_preds, color='red', markers='o', label='Predicted Sales')

plt.xlabel('Country')
plt.ylabel('Total Sales')
plt.title('Actual vs Predicted Sales by Country and Year')

# Adding a legend manually
plt.legend(title='Legend')
plt.xticks(rotation=45) # Rotate x-axis labels for better readability

plt.show()
```

10.1.10.2 Decision Tree

```
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeRegressor
from sklearn.metrics import accuracy_score
from sklearn import tree

# Features and target variable
X = dfNew[['Quantity', 'UnitPrice', 'month', 'year', 'Country', 'Description']] # Features
y = dfNew['Total_Sales'] # Target variable

# Define the preprocessor with both scaling and encoding
preprocessor = ColumnTransformer(
    transformers=[
        ('num', StandardScaler(), ['Quantity', 'UnitPrice', 'month', 'year']),
        ('cat', OneHotEncoder(handle_unknown='ignore'), ['Country', 'Description'])
    ])

# Create the pipeline with preprocessing and model
pipeline_decisionTree = Pipeline(steps=[
    ('preprocessor', preprocessor),
    ('model', DecisionTreeRegressor(max_depth=15, min_samples_split=10, min_samples_leaf=6))
])

#Split the data into training and testing data set
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

#Create and train the Decision Tree model using the training data
pipeline_decisionTree.fit(X_train, y_train)

#Evaluate the model's performance using the testing data

##Make predictions on the test data
y_pred = pipeline_decisionTree.predict(X_test)

#Evaluate the model
mape = mean_absolute_percentage_error(y_test, y_pred)
mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
rmse = sqrt(mse)
r2_decision_tree = r2_score(y_test, y_pred)

#print result
print(f"Mean Absolute Percentage Error (MAPE) = {mape}")
print(f"Mean Absolute Error (MAE) = {mae}")
print(f"Mean Square Error (MSE) = {mse}")
print(f"Root Mean Square Error (RMSE) = {rmse}")
print(f"R2 = {r2_decision_tree}")
```

```

from sklearn.tree import plot_tree

# Extract the trained DecisionTreeRegressor model
decision_tree_model = pipeline_decisionTree.named_steps['model']
# Extract the trained DecisionTreeRegressor model
decision_tree_model = pipeline_decisionTree.named_steps['model']

# Extract feature names from the preprocessor
preprocessor_transformer = pipeline.named_steps['preprocessor']
num_feature_names = preprocessor_transformer.transformers_[0][1].get_feature_names_out()
cat_feature_names = preprocessor_transformer.transformers_[1][1].get_feature_names_out()

# Combine numerical and categorical feature names
feature_names = list(num_feature_names) + list(cat_feature_names)

# Plot the tree result
plt.figure(figsize=(20, 10))
plot_tree(decision_tree_model, feature_names=feature_names, filled=True, rounded=True)
plt.title('Decision Tree Visualization')
plt.show()

```

10.1.10.3 Random Forest

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_squared_error, mean_absolute_error, mean_absolute_percentage_error, r2_score
from math import sqrt

# Assuming dfNew is your DataFrame
# Features and target variable
X = dfNew[['Quantity', 'UnitPrice', 'month', 'year', 'Country', 'Description']] # Features
y = dfNew['Total_Sales'] # Target variable

# Split the data into training and testing data set
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Define the preprocessor with both scaling and encoding
preprocessor = ColumnTransformer(
    transformers=[
        ('num', StandardScaler(), ['Quantity', 'UnitPrice', 'month', 'year']),
        ('cat', OneHotEncoder(handle_unknown='ignore'), ['Country', 'Description'])
    ]
)

# Create the pipeline with preprocessing and RandomForestRegressor
pipeline_rf = Pipeline(steps=[
    ('preprocessor', preprocessor),
    ('model', RandomForestRegressor(max_depth=15, min_samples_split=10, min_samples_leaf=6, random_state=42))
])

# Train the model using the training data
pipeline_rf.fit(X_train, y_train)

# Generate predictions using the test data
y_pred = pipeline_rf.predict(X_test)

# Evaluate the model
mape = mean_absolute_percentage_error(y_test, y_pred)
mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
rmse = sqrt(mse)
r2_random_forest = r2_score(y_test, y_pred)

# Print results
print(f"Mean Absolute Percentage Error (MAPE) = {mape}")
print(f"Mean Absolute Error (MAE) = {mae}")
print(f"Mean Square Error (MSE) = {mse}")
print(f"Root Mean Square Error (RMSE) = {rmse}")
print(f"R² = {r2_random_forest}")
```

```

# Example of new data to predict sales with varying months
new_data = pd.DataFrame({
    'Country': ['United Kingdom'] * 3,
    'Description': ['WHITE HANGING HEART T-LIGHT HOLDER'] * 3,
    'Quantity': [10] * 3,
    'UnitPrice': [2.55] * 3,
    'month': [1, 6, 12], # Different months
    'year': [2022] * 3,
})

# Predict sales using the pipeline
sales_predictions = pipeline_rf.predict(new_data)

# Print predictions
print("Predicted Sales for New Data with Different Months:", sales_predictions)

# Plot the results
plt.figure(figsize=(10, 6))
plt.bar(range(len(sales_predictions)), sales_predictions, tick_label=new_data['month'])
plt.xlabel('Month')
plt.ylabel('Predicted Sales')
plt.title('Predicted Sales for Different Months')
plt.xticks(ticks=range(len(sales_predictions)), labels=new_data['month'])
plt.show()

```

```

# plot result
plt.figure(figsize=(20, 10))
plt.scatter(y_test, y_pred, color='purple')
plt.plot([min(y_test), max(y_test)], [min(y_test), max(y_test)], color='blue', lw = 2)
plt.title('Random Forest Visualization')
plt.xlabel('Actual Values')
plt.ylabel('Predcited Values')
plt.grid(True)
plt.show()

```

10.1.10.4 Compare R²

```

#Data for plotting to compare knn and Linear Regression
models = ['Linear Regression', 'Decision Tree', 'Random Forest']
r2_scores = [r2_linearReg, r2_decision_tree, r2_random_forest]

plt.figure(figsize=(8,6))
plt.bar(models, r2_scores, color=['blue', 'green', 'black'])
plt.xlabel('Model')
plt.ylabel('R2 Score')
plt.title('Comparison of R2 Scores for Linear Regression, Decision Tree and Random Forest')
plt.ylim(0,1) #Set y-axis limits for better visualization
plt.show()

```

